Freie Universität Berlin
Fachbereich für Mathematik und Informatik

An All-Pairs Shortest Path RAPTOR algorithm

studying train shifts and waiting times in an APSP scenario

Master's Thesis

Milan Möllenberg
Matriculation No.: 4360289

Berlin, July 6, 2026

Abstract

An omnipresent problem in schedule-based public transit optimization is the fast and efficient computation of shortest paths across an entire network (All-Pairs Shortest Path, APSP). In cooperation with a major European rail operator, this Master's thesis investigates the impact of temporal relaxations regarding transfer time windows within an APSP scenario. The underlying public transit model is evaluated based on origin-destination time period triples. Given a spatio-temporal discretization of 800 zones and 24 one-hour intervals, this scenario requires a highly efficient algorithm capable of processing up to 15.36 million triples.

To tackle this computational complexity, we present *Stepwise Earliest Raptor-like Path Explorer for Nodes that are Trip-induced* (SERPENT), a trip-based variant of the *Round-Based Public Transit Routing* (RAPTOR). While classical multi-criteria approaches (such as McRAPTOR) incur immense memory and runtime overhead in the APSP context due to exploding Pareto sets, the proposed approach decouples these dependencies. By sequentially executing lean, single-criterion search runs (Earliest Arrival) over chronologically descending starting nodes, our approach efficiently solves a complex bi-criteria problem (travel time optimization for a start time window), similar to the rRAPTOR variant. However, since enforcing a strict maximum waiting time ($\beta < \infty$) leads to a violation of Bellman's principle of optimality for subproblems, our greedy, station-based SERPENT algorithm solves APSP only heuristically. Nevertheless, it provides an exact solution to the problem of locally stepwise shortest paths (Stepwise Earliest APSP, SEAPSP).

Additionally, our approach integrates the consideration of train shifts (ϵ in minutes) to uncover latent transfer potentials that do not yet exist in the current timetable. On a standard consumer notebook, the implementation achieves outstanding performance: even under maximum search complexity ($\epsilon = 5$, $\beta = 60$), the main computation step terminates in just 23.52 seconds, while the one-time initial data preparation requires only about 2 seconds. At the same time, the empirical evaluation demonstrates high solution quality. Despite exploring a limited part of the full search space, SERPENT finds a path of identical length for 71.45% of the optimal APSP paths under strict restrictions ($\epsilon = 0$, $\beta = 15$), and for up to 97.75% under relaxed parameters ($\epsilon = 0$, $\beta = 60$). For future extended analysis, however, the consideration of precisely those origin-destination pairs that cannot be reconstructed via a locally stepwise shortest path is of particular interest. Since it is exactly in these cases that shortest paths impose high waiting times on passengers, this evaluation serves as an ideal, high-performance tool for automated, iterative timetable optimization in large-scale rail networks.

Zusammenfassung

Ein allgegenwärtiges Problem bei der fahrplanbasierten Optimierung des öffentlichen Personenverkehrs ist die schnelle und effiziente Berechnung von kürzesten Pfaden über ein gesamtes Netzwerk (All-Pairs Shortest Path, APSP). Im Rahmen einer Kooperation mit einem großen europäischen Bahnbetreiber untersucht diese Masterarbeit die Auswirkungen von zeitlichen Lockerungen bezüglich der Transferzeitfenster in einem APSP-Szenario. Das zugrunde liegende Verkehrsmodell wird auf der Basis von Start-Ziel-Zeitraum-Tripeln ausgewertet, was bei einer räumlich-zeitlichen Diskretisierung von 800 Zonen und 24 einstündigen Intervallen einen hocheffizienten Algorithmus für bis zu 15,36 Millionen auszuwertende Tripel erfordert.

Zur Bewältigung dieser rechentechnischen Komplexität wird der Algorithmus *Stepwise Earliest Raptor-like Path Explorer for Nodes that are Trip-induced* (SERPENT) vorgestellt, eine trip-basierte Variante des *Round-Based Public Transit Routing* (RAPTOR)-Algorithmus. Während klassische Multi-Kriterien-Ansätze (wie McRAPTOR) aufgrund explodierender Pareto-Mengen immense Speicher- und Laufzeitaufwände im APSP-Kontext verursachen, entkoppelt der vorgestellte Ansatz diese Abhängigkeiten. Durch die sequenzielle Ausführung schlanker, einkriterieller Suchläufe (Früheste Ankunft) über chronologisch absteigend sortierte Startknoten wird ein komplexes Bi-Kriterien-Problem (Reisezeitoptimierung für ein Startzeitfenster) ähnlich zur rRAPTOR-Variante effizient gelöst. Da die Durchsetzung einer strikten maximalen Wartezeit ($\beta < \infty$) jedoch zum Verlust von Bellmans Optimalitätsprinzip für Teilpfade führt, löst unser stationsbasiert gierig arbeitender Algorithmus APSP zwar nur heuristisch, liefert dafür aber eine exakte Lösung des Problems lokal schrittweise kürzester Pfade (Stepwise Earliest APSP, SEAPSP).

Zusätzlich integriert SERPENT die Berücksichtigung von Zugverschiebungen (ϵ in Minuten), um latente Transferpotenziale aufzudecken, die im aktuellen Fahrplan noch nicht existieren. Auf einem standardmäßigen Laptop benötigt die Implementierung eine bemerkenswert kurze Rechendauer: Selbst bei maximaler Suchkomplexität ($\epsilon = 5, \beta = 60$) terminiert der Hauptberechnungsschritt für alle 15,36 Millionen Tripel in nur 23,52 Sekunden, während die einmalige initiale Datenaufbereitung lediglich rund 2 Sekunden beansprucht. Gleichzeitig zeigt die empirische Evaluation eine hohe Ergebnisqualität, bei der SERPENT bereits unter strengen Restriktionen ($\epsilon = 0, \beta = 15$) für 71,45 % und bei gelockerten Parametern ($\epsilon = 0, \beta = 60$) für bis zu 97,75 % der optimalen APSP-Pfade einen Pfad gleicher Länge findet, obwohl nicht der gesamte Suchraum erkundet wird. Interessant für zukünftige weiterführende Auswertungen ist allerdings die Betrachtung genau jener Start-Ziel-Paare, welche mit einem lokal schrittweise kürzesten

Pfad nicht rekonstruiert werden können. Da genau in diesen Fällen kürzeste Pfade dem Fahrgast eine hohe Wartezeit aufbürden, eignet sich die Auswertung ideal als hochperformantes Evaluationswerkzeug für die automatisierte, iterative Fahrplanoptimierung in großflächigen Schienennetzen.

Acknowledgments

I would like to thank Ralf for the opportunity to write my Master's thesis about this engaging topic and contribute to an exciting railway timetabling optimization project. I would also like to thank Boris for his invaluable assistance and productive exchange and my family for their empathic support.

Contents

1 Introduction **8**

1.1 Motivation and Background 8
1.2 Methodological Approach and Goals 9
1.3 Outline of the Thesis 11

2 Literature Review **12**

2.1 Classical Foundations of Shortest Path Routing 12
2.2 Time-Dependent and Multi-Criteria Routing 12
2.3 Advanced Public Transit Routing Algorithms 13
2.4 Algorithmic Customization and Structural Challenges 14

3 Mathematical Formulation **15**

3.1 Basic Graph Notations 15
3.2 Specific Nodes and Arcs 17
3.3 Train Shifts 20
3.4 Trip Segments and OD paths 22

4 Problem Definitions and Algorithmic Search Spaces **25**

4.1 Dominance and APSP 25
4.2 Search Space 27
4.3 Sub-path Optimality and Algorithmic Restrictions 27

5 Algorithm **34**

5.1 RAPTOR Algorithm 34
5.2 SERPENT Algorithm 37
5.3 Pruning and Limitations 39
5.4 Possible Extensions 41
5.5 Algorithmic Advantages 42
5.6 Time Complexity 44

6 Experimental Results **47**
6.1 Input data 47
6.2 Experimental Runtime 48
6.3 Average Travel Time 50
6.4 Comparison to other Algorithms 51

7 Conclusion and Outlook **60**
7.1 Summary of Contributions 60
7.2 Discussion of Path Quality and Structural Insights 61

Bibliography **62**

Declaration of Independent Work **64**

1 Introduction

1.1 Motivation and Background

In the field of algorithmic graph theory and operations research, shortest path algorithms have traditionally been established as computational powerhouses. While their early development in the 1950s was deeply rooted in abstract graph theory and logistical optimization for military operations [12, 3], as well as demonstrating the capabilities of early computing hardware [9], their mathematical versatility quickly led to widespread adoption across a diverse spectrum of modern fields. Today, applications of shortest path paradigms are omnipresent, utilized in network routing protocols to direct data packets efficiently across the internet [23, 19], in robotic motion planning to navigate obstacles in dynamic environments [15], in financial systems for multi-currency arbitrage detection [1], and even in computational biology for sequence alignment and metabolic pathway analysis [21, 22].

Although most foundational models were mathematically generalized for arbitrary graphs, physical transportation and infrastructure topologies rapidly emerged as one of their most prominent and intuitive real-world applications. However, among these, schedule-based public transit optimization remains one of the most structurally challenging domains due to its inherently time-dependent nature.

In cooperation with a large European passenger railway operator, a dedicated research project at the Zuse Institute Berlin (ZIB) focused on the optimization of real-world public transit timetables. A primary objective of this project was to analyze and improve passenger transfer connections. If the waiting time during transfers is excessively long, the attractiveness of public transit drops significantly, causing passengers to prefer individual vehicular transit or other alternatives. By shifting trip departure times by just a few minutes, essential transfer possibilities can either be enabled or disabled, heavily influencing the average travel experience across the entire network.

As the scope of such transit optimization models widens to capture realistic passenger behavior, the underlying computational complexity escalates rapidly. Incorporating dynamic aspects such as variations in passenger demand, the necessity of tracking multiple alternative paths, and enforcing strict constraints on transfer windows causes the runtime and memory requirements of

traditional graph-routing algorithms to grow exponentially [24]. Consequently, when planners attempt to perform iterative modifications to a massive timetable, evaluating the systemic utility of these adjustments becomes a challenge. This computational bottleneck becomes particularly severe when the evaluation cannot be restricted to a few high-demand connections, but must instead cover the entire network to ensure full structural coverage, including low-frequency and rural services.

Therefore, a critical need has emerged for a high-performance routing algorithm. It should be capable of computing the All-Pairs Shortest Path (APSP) problem across an entire timetable exceptionally fast and without requiring extensive, time-consuming preprocessing phases. In the context of iterative timetable optimization and rapid re-optimization loops, minimizing preprocessing overhead and maximizing query speed are essential.

To evaluate the systemic utility of these timetable adjustments across a massive real-world infrastructure, an algorithm must process millions of origin-destination combinations. In this work, the underlying passenger demand structure is modeled using fine-grained Origin-Destination-Period triples rather than static location pairs. By partitioning the day into distinct time intervals and inducing a spatial family of zones (where each physical station is mapped to one or more geographical zones), we define an origin node for each unique zone-period pair and a distinct destination node for each zone. For our empirical evaluation, we utilize 24 individual one-hour intervals across a full 24-hour cycle and 800 distinct zones. This high-resolution spatio-temporal discretization accurately mirrors the time-dependent nature of real-world transit networks, requiring our routing engine to efficiently explore paths for up to 15.36 million zone-to-zone-within-hour triples under highly restricted time windows.

1.2 Methodological Approach and Goals

To address this challenge, my thesis presents an approach customized to our timetable framework for the APSP scenario on large-scale public transit networks. We utilize a Single-Source Shortest Path (SSSP) idea for every source node to construct the complete APSP solution matrix.

The algorithmic core of our *Stepwise Earliest Raptor-like Path Explorer for Nodes that are Trip-induced* (SERPENT) is heavily inspired by the Round-Based Public Transit Routing (RAPTOR) algorithm introduced by Delling, Pajor, and Werneck [6]. For a fixed origin station and start time, RAPTOR finds shortest paths to all stations with respect to earliest arrival time. RAPTOR differs fundamentally from classical Dijkstra-based architectures. It bypasses the computational sorting barrier that typically hampers Dijkstra-based labeling approaches in dense networks.

Our variant adopts this round-based label-correction approach to iteratively update station-

based arrival labels with the best accessibility. However, a major distinction of our underlying timetable data is that trips are entirely independent of any fixed assignment to predefined routes or lines. Consequently, the primary exploration step in our framework is fundamentally trip-based rather than route-based.

Furthermore, realistic public transit optimization typically demands evaluating journeys based on total travel time rather than merely minimizing the earliest arrival time for a single predefined departure. While multi-criteria extensions such as McRAPTOR manage full Pareto-optimal sets of label bags at every stop to minimize conflicting criteria simultaneously, maintaining these non-dominated sets introduces an exponential growth in state space. When scaled to an APSP scenario, this characteristic leads to a prohibitive memory footprint and severe execution delays [cf. 6, Section 5.3]. To circumvent this computational barrier, our approach decouples the multi-criteria dependency. Instead of evaluating heavy Pareto-optimal label bags, we execute a single-criterion core search iteratively across numerous source configurations. This allows our framework to effectively optimize total travel time through a succession of lean, arrival-time-based RAPTOR passes, thereby drastically reducing memory consumption and maximizing algorithmic throughput.

However, executing thousands of independent single-criterion searches across a large-scale infrastructure would inherently trigger a massive computational overhead. To shift from such a widespread, breadth-first exploration to a lean and highly directed search, SERPENT integrates a station-based pruning mechanism. Yet, because our model enforces strict bounds on maximum tolerable waiting times to reflect realistic passenger behavior, a structural challenge arises: bounding the localized transfer windows violates the classical First-In-First-Out (FIFO) property, which states that "later departures cannot lead to earlier arrivals" [2]. Therefore, our greedy pruning mechanism, when applied to a timetable with restricted maximum waiting times, disrupts Bellman's subpath optimality principle. Consequently, while our framework shifts to a heuristic search model regarding the unconstrained APSP formulation, it remains mathematically exact for the restricted domain of locally stepwise earliest paths.

A highly specialized feature of our work is the integration of train shifts into the path exploration phase. SERPENT considers additional, latent transfer opportunities that do not exist in the current schedule but could be unlocked if the timestamps of an entire trip were cleverly and slightly adjusted (shifted). The paths discovered by SERPENT provide a crucial mathematical upper bound on the maximum potential benefit that a global timetable shift could achieve for passenger connectivity. Furthermore, they serve as an invaluable quality control benchmark for even more complex, exact optimization algorithms.

By employing a heuristic pruning strategy that deliberately skips paths that are only achievable through excessively long transfer times, SERPENT remains computationally lightweight. Due to its fast runtime, the algorithm can pinpoint specific origin-destination pairs (OD pairs) whose

transfer connections are sub-optimal in the current timetable. This makes our framework uniquely suited as an evaluation engine for the iterative, automated improvement of transfer synchronization in large-scale railway networks. While this thesis provides the foundation for these optimization methods, substantial research on iterative timetable improvements remains for future work.

1.3 Outline of the Thesis

The remainder of this thesis is structured as follows. Chapter 2 provides a comprehensive review of the existing literature, tracing the evolution from classical graph-based foundations to state-of-the-art public transit routing paradigms. Chapter 3 formalizes the public transit network, defining the exact notation for timetables and relaxed transfer arcs. The next Chapter 4 defines the All-Pairs Shortest Path problem (APSP) on zones and explains why it does not suffice for our restricted waiting time and pruning mechanisms. We therefore define the Stepwise Earliest APSP problem (SEAPSP). In Chapter 5, the technical details of our trip-based RAPTOR variant, the sequential sorting of emanation nodes, and the self-pruning mechanisms are detailed and algorithmically formulated. Chapter 6 presents the computational results and runtime evaluations using real-world railway datasets, quantifying the trade-offs between heuristic acceleration and solution quality. Finally, Chapter 7 concludes the thesis with a summary of our findings and discusses prospective avenues for future research.

2 Literature Review

The computation of optimal journeys in transportation networks is a core problem in algorithmic graph theory and operations research. This Chapter reviews the existing literature on shortest path algorithms, beginning with classical foundations and progressing toward state-of-the-art multi-criteria techniques optimized specifically for schedule-based public transit networks.

2.1 Classical Foundations of Shortest Path Routing

The algorithmic foundations for routing problems date back to the mid-20th century. For Single-Source Shortest Path (SSSP) computations on general graphs with non-negative edge weights, the label-setting algorithm by Dijkstra [9] remains the seminal reference work. To address graphs containing negative edge weights without negative cycles, the label-correcting approaches by Bellman [3] and Ford [12] provide necessary exact guarantees.

When analyzing All-Pairs Shortest Path (APSP) variations across large-scale infrastructure, the density of the underlying network significantly impacts performance. Johnson [14] introduced an efficient algorithm tailored specifically for sparse networks. By utilizing a re-weighting technique based on the Bellman-Ford algorithm, Johnson's approach allows the subsequent application of Dijkstra's algorithm from every vertex. When combined with modern heap structures, this achieves a time complexity of $O(n^2 \log n)$ on sparse topologies, where n denotes the number of vertices. In scenarios where decisions and constraints must adapt dynamically during the path-exploration phase or require structured state-space tracking, concepts from optimal control and system feedback mechanisms, such as those discussed by Kirk [16], provide essential foundational frameworks for managing sequential decision-making topologies.

2.2 Time-Dependent and Multi-Criteria Routing

Public transit networks fundamentally differ from static road networks due to their inherently schedule-based, time-dependent nature. Elements of time-dependent pathfinding were formalized early on, with Nachtigall [20] analyzing shortest path problems with specific applications to railway optimization and periodic schedule integration, evaluating how travel duration and

waiting times vary dynamically based on the departure timestamp.

Furthermore, realistic transit routing rarely depends on a single criterion such as pure travel time. Passengers typically optimize across multiple conflicting dimensions, most notably total journey duration and the number of transfers. The multi-objective shortest path (MOSP) problem aims to identify the complete set of non-dominated Pareto-optimal paths. Martins [18] established a highly influential label-setting algorithm generalized for multi-criteria shortest path problems, which serves as a classical theoretical benchmark. However, as noted extensively in modern literature surveys such as the comprehensive overview by Bast et al. [2], classical multi-criteria labeling algorithms often suffer from high computational overhead and extensive memory consumption when scaled to large, continent-scale transit systems due to the exponential growth of non-dominated label sets at each node.

2.3 Advanced Public Transit Routing Algorithms

To overcome the efficiency barriers of Dijkstra-based labeling in public transit networks, several specialized non-Dijkstra paradigms have emerged over the last two decades.

A prominent milestone is the Round-Based Public Transit Routing (RAPTOR) algorithm introduced by Delling, Pajor, and Werneck [6]. Unlike traditional graph-search approaches, RAPTOR operates directly on the timetable data structures (routes, trips, and stops) rather than a graph representation. By operating in discrete rounds–where round k computes the earliest arrival times using at most $k - 1$ transfers–RAPTOR avoids the necessity of maintaining a priority queue, eliminating a major computational sorting barrier and significantly accelerating query times.

To further optimize queries, alternative non-graph paradigms have focused on scanning arrays or individual vehicle configurations directly. Prominent among these is the Connection Scan Algorithm (CSA) by Dibbelt et al. [8], which processes an ordered array of elementary connections linearly, requiring virtually no preprocessing. CSA bypasses traditional graph traversals entirely by sorting elementary connections chronologically and scanning them linearly in a single pass. While structurally simple, adapting CSA to multi-criteria queries requires maintaining multiple fractional labels, which can impair its performance advantage in dense network profiles. Conversely, the Trip-Based Public Transit Routing algorithm developed by Witt [25] shifts the focus from stops or connections onto the individual vehicle trips themselves, evaluating transfers directly between intersecting trips to achieve extremely rapid query phases.

2.4 Algorithmic Customization and Structural Challenges

A major challenge in contemporary routing engines is minimizing both preprocessing overhead and query execution time for single-source methods deployed iteratively across all pairs. While standard multi-criteria variants like McRAPTOR, also introduced by Delling, Pajor, and Werneck [6] track full Pareto sets of labels to ensure mathematical exactness, these lead to a considerable memory footprint and slower throughput during large APSP passes.

To bypass these scaling limitations, modern high-performance engines often adopt single-criterion core searches inspired by the round-based structural decomposition of RAPTOR. However, enforcing deterministic operational constraints–such as capping the maximum tolerable waiting time to preserve realistic passenger behavior–introduces structural deviations. Specifically, bounding localized transfer windows or applying strict pruning heuristics can cause a violation of the classical First-In-First-Out (FIFO) property and disrupt Bellman's subpath optimality principle. Under these non-FIFO conditions, a subpath of a Pareto-optimal journey is no longer guaranteed to be optimal itself. While the specific constraint of bounded waiting times is a feature of modern transit engines, the foundational theoretical underpinnings and mathematical pitfalls of non-FIFO time-dependent networks–and the resulting breakdown of Bellman's principle–were structurally classified early on by Nachtigall [20].

Addressing these challenges while circumventing the sorting overhead of standard priority queues forms a core component of recent research, leading to new algorithmic boundaries in eliminating structural bottlenecks. In a recent study, Duan et al. [10] developed a directed Single-Source Shortest Path (SSSP) algorithm with a complexity below the long-standing sorting barrier of $O(m + n \log n)$ (n denotes the number of vertices, m represents the number of edges) that classical priority-queue-based implementations suffer from. On sparse graphs, where $m \in O(n)$, their approach applied for APSP achieves a time complexity of $O(n^2 \log^{2/3} n)$. It should be noted, however, that this improvement remains strictly theoretical. While it shifts the fundamental bounds of computational complexity, the substantial hidden constants involved prevent it from providing a practical speedup in real-world application scenarios.

3 Mathematical Formulation

Our formulation relies on a time-expanded network structure. Unlike time-dependent models where edge weights are functions of time, our graph explicitly duplicates nodes for each scheduled event, assigning a fixed timestamp $\tau(v)$ to each node v. For readability reasons within our project we share and adapt some notation with the work of Maristany de las Casas [17] and the master thesis of Hommel [13]. First we will explain basic mathematical expressions used in our thesis. Then we will describe how we build our directed graph:

$$G = G(\epsilon, \beta, h) = (\mathcal{N}, \mathcal{A}),$$

consisting of the node set:

$$\mathcal{N} = \mathcal{N}_{\text{start}} \cup \mathcal{N}_{\text{trip}} \cup \mathcal{N}_{\text{end}}$$

and weighted arcs:

$$\mathcal{A}(\epsilon, \beta, h) = \mathcal{A}_{\text{board}}(h) \cup \mathcal{A}_{\text{travel}} \cup \mathcal{A}_{\text{dwell}} \cup \mathcal{A}_{\text{trans}}(\epsilon, \beta) \cup \mathcal{A}_{\text{alight}}.$$

G is the short notation of $G(\epsilon, \beta, h)$, but is always dependent on the parameter choice. The long version will be used if we want to remind the reader on its dependence. This graph will be explored by our algorithm SERPENT. The resulting paths indicate how well zones are connected (or rather in case $\epsilon \geq 0$ *might become* connected) at starting time period h.

3.1 Basic Graph Notations

Definition 3.1. A *node* in general stands for an anticipated or possible event in a public transit network. Let $\mathcal{N}$ denote the union of all considered event nodes. Function

$$\tau : \mathcal{N} \to \mathbb{R}^+$$

represents the timestamp of a node $v \in \mathcal{N}$. As our input timetable represents all trips within one day, if not otherwise identified, $\tau(v)$ stands for elapsed seconds since midnight. An *arc* (u, v) is an ordered pair of nodes u and v, where u is called the *tail* and v is called the *head* of (u, v). The

union of all arcs is denoted as $\mathcal{A}$. The *weight* $w((u,v))$ of an arc is defined as the difference $\tau(v) - \tau(u)$. Later on, we will use a handy notation for outgoing and incoming neighbour nodes used similarly by Maristany de las Casas [17]:

$$\delta^+(u) \coloneqq \{v \mid (u,v) \in \mathcal{A}\},$$

$$\delta^-(v) \coloneqq \{u \mid (u,v) \in \mathcal{A}\}.$$

Note that in our thesis this notation is used for sets of *nodes* instead of *arcs* . Further details on arcs follow in the next Section 3.2. A *path* is an ordered sequence of nodes in $\mathcal{N}$:

$$P = (v_0, \ldots, v_k, \ldots, v_{n-1}).$$

A path P is called *valid* if for all k with $0 < k < n$ the following holds:

$$(v_{k-1}, v_k) \in \mathcal{A}.$$

Additionally, we write $(v_{k-1}, v_k) \in P$ for arcs connecting consecutive nodes in P. We call P *simple* [1] if for every node $v_k \in P$ there is exactly one outgoing arc in P for $0 \leq k < n-1$ and one incoming arc in P for $0 < k \leq n-1$. Let $\mathcal{S}$ be the set of public transit *stations*. A *zone* z is a subset of $\mathcal{S}$, which can be chosen to represent a certain region or urban quarter. Zones do not have to be disjoint with each other. A collection of all zones is called $\mathcal{Z}$. Let

$$(s_1, \ldots, s_n), \quad s_i \in \mathcal{S} \; \forall \; i \leq n$$

be a sequence of stations. Each train operating on such a sequence induces a *trip*, alternating between *departure* and *arrival nodes*. In most public transit networks, trips are aggregated into routes and operate mostly periodically. In this paper, neither aggregation nor periodicity is required. An *arrival node* is the event of a certain trip i arriving at station $s \in \mathcal{S}$. Consequently, a *departure node* is the event of trip i departing from station $s \in \mathcal{S}$:

$$d^i_\sigma \text{ is the } \textit{departure} \text{ event of trip } i \text{ at index } 0 \leq \sigma < |i|,$$

$$a^i_\sigma \text{ is the } \textit{arrival} \text{ event of trip } i \text{ at index } 0 < \sigma \leq |i|.$$

For better readability from now on we always assume σ to be within its boundary whenever we write d^i_σ or a^i_σ. We also reduce the notation to a^i and d^i whenever the exact index σ is not relevant. The union of all normal stop arrival and departure nodes is called $\mathcal{N}_{\text{trip}}$.

[1] If not explicitly stated otherwise, we assume paths to be valid and simple from now on

Definition 3.2 (Trip)**.** We define *trip i* as a path of stop event nodes v visited by an individual train, alternating between departure and arrival, serving $|i|$ stations:

$$i = \left(d_0^i, a_1^i, d_1^i, \ldots, a_\sigma^i, d_\sigma^i, \ldots, a_{|i|}^i\right) \in \mathcal{N}_{\text{trip}}^{2|i|}.$$

The station visited by node v will be written as $\mathsf{st}(v)$, $v \in \mathcal{N}_{\text{trip}}$. Note that all trips start with a single departure node d_0^i and end with a arrival node $a_{|i|}^i$. In between there are pairs of arrival and departure nodes, coming to and leaving from the same station:

$$\mathsf{st}\left(a_\sigma^i\right) = \mathsf{st}\left(d_\sigma^i\right) \quad \forall\, 0 < \sigma < |i|.$$

Therefore, any trip i always contains an even number of nodes (exactly $2|i|$) and visits $|i|$ not necessarily distinct stations. We denote the set of all trips $\mathcal{I}$. We assume that arrival and departure times increase monotonously along each trip:

$$\tau\left(a_\sigma^i\right) \leq \tau\left(d_\sigma^i\right) \quad \text{and} \quad \tau\left(d_\sigma^i\right) \leq \tau\left(a_{\sigma'}^i\right) \quad \forall\, \sigma < \sigma'.$$

3.2 Specific Nodes and Arcs

This Section explains which specific nodes will be included in our node set $\mathcal{N}$ and which arcs will be distinguished in our arc set $\mathcal{A}$.

Definition 3.3 (Artificial Nodes)**.** We want to plot and evaluate our results starting from each starting zone z. Thus we define our *artificial start node* set:

$$\mathcal{N}_{\text{start}} \coloneqq \left\{d_z^{\text{art}} \mid z \in \mathcal{Z}\right\},$$

containing one artificial start node d_z^{art} for each zone. It represents the intention to start a journey from the centre of zone z. For the end of our paths we use the following *artificial end nodes*:

$$\mathcal{N}_{\text{end}} \coloneqq \left\{a_z^{\text{art}} \mid z \in \mathcal{Z}\right\}.$$

We define the vertex set $\mathcal{N}$ of our directed graph G as the union of these nodes:

$$\mathcal{N} = \mathcal{N}_{\text{start}} \cup \mathcal{N}_{\text{trip}} \cup \mathcal{N}_{\text{end}}.$$

For the remainder of this thesis, whenever we refer to an origin-destination pair (OD pair) of zones $(z, z') \in \mathcal{Z} \times \mathcal{Z}$, it is implicitly and formally understood as a graph routing query seeking

a valid path in G from the specific artificial departure node $d_z^{\text{art}} \in \mathcal{N}_{\text{start}}$ to the artificial arrival node $a_{z'}^{\text{art}} \in \mathcal{N}_{\text{end}}$.

Definition 3.4 (Travel Arcs, Dwell Arcs)**.** As we have different kind of nodes, we also need to distinguish arcs. The first two kinds, *travel arcs* $\mathcal{A}_{\text{travel}}$ and *dwell arcs* $\mathcal{A}_{\text{dwell}}$, are contained within the trips. An arc in $\mathcal{A}_{\text{travel}}$ represents travelling with the trip from one station to the next while an arc in $\mathcal{A}_{\text{dwell}}$ can be understood as the waiting time of the train at a station before it departs again:

$$\mathcal{A}_{\text{travel}} \coloneqq \left\{ (d_\sigma^i, a_{\sigma+1}^i) \in \mathcal{N}_{\text{trip}}^2 \mid i \in \mathcal{I}, 0 \leq \sigma < |i| \right\}, \tag{3.1}$$

$$\mathcal{A}_{\text{dwell}} \coloneqq \left\{ (a_\sigma^i, d_\sigma^i) \in \mathcal{N}_{\text{trip}}^2 \mid i \in \mathcal{I}, 0 < \sigma < |i| \right\}. \tag{3.2}$$

Note that the arcs within i alternate, beginning and ending with a travel arc. Each arc in $\mathcal{A}_{\text{travel}} \cup \mathcal{A}_{\text{dwell}}$ is dedicated exclusively for its trip.

The third kind are arcs required to change the trip. Their duration allows for passengers to leave trip i and board trip j. To include these arcs in $\mathcal{A}$ would not make sense for an arbitrary pair of trips, but they do between those time-wise and geographically close to each other. In order to enforce time-wise proximity of trips we introduce the following variable:

Definition 3.5 (Maximum Waiting Time, Minimum Transfer Time)**.** We define $\beta \in \mathbb{R}^+$ to be a global *maximum allowed waiting time*. Any transfers that include a waiting time longer than β are considered undesirable by passengers and get excluded from our arc set A. Later in the computational Chapter 6 of our thesis we will use different input data sets for $\beta \in \{15, 30, 45, 60\}$ minutes.

We also need to introduce a symmetric *minimum transfer time* function walk. It will serve as a lower bound on the time difference between nodes a^i and d^j:

$$\mathsf{walk} : (\mathcal{N} \times \mathcal{N}) \to \mathbb{R}^+.$$

For $(\mathcal{N}_{\text{trip}} \times \mathcal{N}_{\text{trip}})$, the magnitude of $\mathsf{walk}(a^i, d^j)$ depends mostly on the nodes belonging to the same station or not. If both nodes serve the same station s, it represents the time one needs to change platforms at s. For different $\mathsf{st}(u) = s$, $\mathsf{st}(v) = s'$ it represents the average walking time between stations s and s'. For the artificial start nodes $\mathcal{N}_{\text{start}}$ and end nodes $\mathcal{N}_{\text{end}}$, $\mathsf{walk}(d_z^{\text{art}}, d)$ and $\mathsf{walk}(a, d_z^{\text{art}})$ represents the time needed to walk between the centre of zone z and the station of node d, a, respectively :

We reuse the name of this function intuitively to shorten notation:

$$\mathsf{walk}(z, d) \coloneqq \mathsf{walk}(d_z^{\text{art}}, d),$$

$$\mathsf{walk}(a, z) \coloneqq \mathsf{walk}(a, d_z^{\text{art}}).$$

The degree of accuracy can be improved if walk is not only dependent on stations, but also on the platform of respective nodes. Walking towards a distant platform of a crowded station during rush hours can take a long time. For this paper we expect the walk-function to be given.

Definition 3.6 (Transfer Arcs)**.** To account for potential adjustments in the timetable, the set of *transfer arcs* is parametrized by both a maximum permissible timetable deviation $\epsilon \geq 0$ and the maximum allowed waiting time β. We define the set:

$$\mathcal{A}_{\text{trans}}(\epsilon, \beta) \coloneqq \left\{(a^i, d^j) \in \mathcal{N}_{\text{trip}}^2 \mid -2\epsilon \leq \tau\big(d^j\big) - \tau\big(a^i\big) - \mathsf{walk}(a^i, d^j) \leq \beta + 2\epsilon, \quad i \neq j\right\}. \tag{3.3}$$

For now assume $\epsilon = 0$, further explanation on ϵ follow in the next Section 3.3. Transfer times for arcs in this set are long enough to be manageable for passengers but not excessively long, we also call transfers *feasible* if their transfer nodes form an arc in $\mathcal{A}_{\text{trans}}(\epsilon, \beta)$. We do not require additional criteria to ensure that arrival and departure operate locally close to each other. This is already covered by the bounds, since for stations being further apart, we require a walking time walk much greater than β.

These arcs suffice to describe a railway path from one station to another. However, in order to involve our set of artificial start $\mathcal{N}_{\text{start}}$ and end nodes $\mathcal{N}_{\text{end}}$ we need to add two footpath versions of arcs. Let a be an arrival and d a departure node in $\mathcal{N}_{\text{trip}}$. We use the notation $\tau\big(z, d\big)$ for the timestamp a passenger needs to start walking from the centre of zone z to reach the departure node d in time and $\tau\big(a, z\big)$ as the timestamp of expected arrival when arriving at a and walking to the centre of zone z:

$$\tau\big(z, d\big) \coloneqq \tau\big(d\big) - \mathsf{walk}(z, d),$$

$$\tau\big(a, z\big) \coloneqq \tau\big(a\big) + \mathsf{walk}(a, z).$$

To model reasonable pedestrian access, we define a maximum permissible walking duration $W \in \mathbb{R}^+$. We also introduce a set of disjunct *time periods* h with $\dot{\bigcup} h = \mathcal{H}$ (in Chapter 6 we chose 24 time windows of one hour each). To connect an artificial start node d_z^{art} to all departure nodes reachable by foot from the centre of z (when beginning to walk during time period $h \in \mathcal{H}$), we introduce the set of *boarding arcs*:

$$\mathcal{A}_{\text{board}}(h) \coloneqq \left\{(d_z^{\text{art}}, d) \in \mathcal{N}_{\text{start}} \times \mathcal{N}_{\text{trip}} \mid \mathsf{walk}(z, d) \leq W, \quad \tau\big(z, d\big) \in h\right\}. \tag{3.4}$$

Similarly, the set of *alighting arcs* is defined as:

$$\mathcal{A}_{\text{alight}} \coloneqq \left\{(a, a_z^{\text{art}}) \in \mathcal{N}_{\text{trip}} \times \mathcal{N}_{\text{end}} \mid \mathsf{walk}(a, z) \leq W\right\}, \tag{3.5}$$

which connects arrival nodes to all destination zones within the acceptable walking threshold W.

The full arc set of our graph G is concluded as:

$$\mathcal{A}(\epsilon, \beta, h) \coloneqq \mathcal{A}_{\text{board}}(h) \cup \mathcal{A}_{\text{travel}} \cup \mathcal{A}_{\text{dwell}} \cup \mathcal{A}_{\text{trans}}(\epsilon, \beta) \cup \mathcal{A}_{\text{alight}}. \tag{3.6}$$

The weight of our arcs (u, v) is defined to match our intuition as the time difference between event node u and v:

$$w(u, v) = \begin{cases} \tau(v) - \tau(u), & \text{if } (u, v) \in \mathcal{A}_{\text{trans}} \cup \mathcal{A}_{\text{travel}}, \\ \mathsf{walk}(u, v), & \text{if } (u, v) \in \mathcal{A}_{\text{alight}} \cup \mathcal{A}_{\text{board}}(h). \end{cases}$$

Remark 1. *Given the piecewise defined arc weights above, the total* weight *of a valid path $P = (v_0, \ldots, v_{n-1})$ is formally defined as the sum of its constituent arc weights:*

$$w(P) \coloneqq \sum_{(u,v)\in P} w(u, v).$$

It is worth noting that explicit tracking and summation of individual arc weights along a path is practically redundant for travel, dwell, and transfer segments. Because the weights for arcs in $\mathcal{A}_{trans} \cup \mathcal{A}_{travel} \cup \mathcal{A}_{dwell}$ are strictly defined as the direct timestamp differences $\tau(v_k) - \tau(v_{k-1})$, the summation telescopes mathematically for any pure railway segment:

$$\sum_{(u,v)\in P_{rail}} w(u, v) = \tau(v_{end}) - \tau(v_{start}).$$

Consequently, any two valid sub-paths operating between the same two event nodes $u, v \in \mathcal{N}$ are inherently guaranteed to yield the exact same path weight. In our algorithmic implementation, this property allows us to evaluate and track the journey duration solely via a direct comparison of node timestamps rather than accumulating arc values, which significantly increases computational efficiency.

3.3 Train Shifts

Since we want to study the effect of train shifts, we want to include even more arcs into $\mathcal{A}_{\text{trans}}$. As motivated in the definition of $\mathcal{A}_{\text{trans}}(\epsilon, \beta)$, choosing $\epsilon > 0$ relaxes the temporal boundaries of transfers. This relaxation anticipates the effect of shifting entire trips up to ϵ minutes compared to the original choice of arrival and departure times. Let $\epsilon \in \mathbb{N}^+$ be a maximum allowed time

shift in minutes. Remember from Section 3.2:

$$\mathcal{A}_{\text{trans}}(\epsilon, \beta) := \left\{ (a^i, d^j) \in \mathcal{N}_{\text{trip}}^2 \mid -2\epsilon \leq \tau\left(d^j\right) - \tau\left(a^i\right) - \mathsf{walk}(a^i, d^j) \leq \beta + 2\epsilon, \quad i \neq j \right\}.$$

Note that for bigger ϵ, the transfer arc increases in size, i.e., for $\epsilon < \epsilon'$:

$$\mathcal{A}_{\text{trans}}(\epsilon, \beta) \subseteq \mathcal{A}_{\text{trans}}(\beta, \epsilon').$$

Let a^i and d^j be stop events such that $(a^i, d^j) \notin \mathcal{A}_{\text{trans}}(\beta, 0)$. If the connection got omitted due to a small violation of the lower bound, we can shift i to the left and j to the right, each up to ϵ minutes. This results in a bigger time window between a^i and d^j. If a small violation of the upper bound prevented the connection, we shift in the opposite direction to slightly decrease the time difference between a^i and d^j below β.

Definition 3.7 (Shift Assignment)**.** Formally, a *shift assignment* for a trip $i \in \mathcal{I}$ is defined as an integer scalar $t_i \in [-\epsilon, \epsilon]$, which represents the uniform time shift applied to the entire trip in minutes. A global *timetable shift* is a collection of shift assignments for all trips, denoted by vector $t = (t_i)_{i \in \mathcal{I}}$. Applying this shift assignment alters all original event timestamps τ of each trip i to their shifted timestamps τ^t as follows:

$$\begin{aligned} \tau^t\left(d^i_\sigma\right), &:= \tau\left(d^i_\sigma\right) + t_i \quad 0 \leq \sigma < |i|, \\ \tau^t\left(a^i_\sigma\right), &:= \tau\left(a^i_\sigma\right) + t_i \quad 0 < \sigma \leq |i|. \end{aligned}$$

Note that shifting both all arrivals and all departures of trip i by the exact same scalar t_i ensures that the internal structure of the trip (specifically all travel times between stations and all dwell times at stations) remains completely invariant under any valid shift assignment. Under a specific shift assignment t, a transfer $(a^i_\sigma, d^j_{\sigma'})$ from trip i to j is feasible if and only if it satisfies the baseline requirements:

$$0 \leq \tau^t\left(d^j_{\sigma'}\right) - \tau^t\left(a^i_\sigma\right) - \mathsf{walk}(a^i_\sigma, d^j_{\sigma'}) \leq \beta.$$

Substituting the shifted timestamps reveals the relation to our relaxed transition arc set:

$$\tau^t\left(d^j_{\sigma'}\right) - \tau^t\left(a^i_\sigma\right) = \left(\tau\left(d^j_{\sigma'}\right) - \tau\left(a^i_\sigma\right)\right) + \left(t_j - t_i\right).$$

Since $t_i, t_j \in [-\epsilon, \epsilon]$, the maximum possible adjustment of the time window is bounded by $(t_j - t_i) \in [-2\epsilon, 2\epsilon]$, explaining the usage of 2ϵ during the definition of transfer arcs $\mathcal{A}_{\text{trans}}$. The set $\mathcal{A}_{\text{trans}}(\epsilon, \beta)$ contains all transfer arcs that could *potentially* become feasible under at least one valid global shift assignment t.

In this thesis, we do not fix a specific global assignment t. Paths routed through $G(\epsilon, \beta, h)$ represent a mathematical upper bound on the maximum potential benefit that a timetable shift could achieve for passenger connectivity. The graph $G(\epsilon, \beta, h)$ acts as a structural envelope with loosened constraints, allowing us to evaluate the potential benefit without fixing individual shift decisions. For each OD pair, we evaluate an optimal path under the assumption that underlying timetable shifts will be configured in its favour. Note that if we choose the null vector for each t_i (the only choice for $\epsilon = 0$), we operate on the unaltered currently existing timetable. The arcs always depend on β and ϵ, but for better readability we will also write $\mathcal{A}$ and $\mathcal{A}_{\text{trans}}$ instead of $\mathcal{A}(\epsilon, \beta)$ and $\mathcal{A}_{\text{trans}}(\epsilon, \beta)$ whenever the specification is not needed.

Ultimately, we outline which structures are enveloped in our definition of a *timetable*:

Definition 3.8 (Timetable)**.** A *timetable* $\mathcal{T}$ of a public transit network is a tuple

$$\mathcal{T} = (\mathcal{S}, \mathcal{N}_{\text{trip}}, \mathcal{A}(\epsilon, \beta, h), \tau, \mathcal{Z}, \mathcal{H}),$$

representing a specific choice for each of its elements:

$\mathcal{S}$	set of all stations,
$\mathcal{N}_{\text{trip}}$	set of all arrival and departure nodes,
$\mathcal{A}(\epsilon, \beta, h)$	set of all arcs depending on ϵ, β, h,
ϵ	maximum allowed time shift,
β	maximum waiting time,
τ	timestamps for each node in $\mathcal{N}_{\text{trip}}$,
$\mathcal{Z}$	family of subsets of $\mathcal{S}$,
$\mathcal{H}$	set of disjunct time periods h.

Additionally, since every node in $\mathcal{N}_{\text{trip}}$ is associated with a trip, $\mathcal{T}$ implicitly defines the set of all trips $\mathcal{I}$, which partition and structure the event nodes in $\mathcal{N}_{\text{trip}}$. For ease of notation, we consider $\mathcal{I}$ an integral part of the timetable's scope.

3.4 Trip Segments and OD paths

A *trip* represents the complete operational run of a vehicle in a public transit network from its initial to its terminal station. A partial run, where a passenger boards a trip at one station and alights at another station, is called a *trip segment*. A journey or path through the network is then modeled a concatenation of several trip segments. While trips in reality often operate

periodically on fixed lines or routes (e.g., every 20 minutes), our algorithmic framework does not require trips to be aggregated, grouped, or associated with any specific line structure. Every trip is treated as entirely independent.

Definition 3.9 (Trip Segment)**.** Let $i \in \mathcal{I}$ be a trip defined by its sequence of stop event nodes:

$$i = \left(d_0^i, a_1^i, d_1^i, \ldots, a_\sigma^i, d_\sigma^i, \ldots, a_{|i|}^i\right) \in \mathcal{N}_{\text{trip}}^{2|i|}.$$

A *trip segment* $\gamma = (i, \sigma^{\text{start}}, \sigma^{\text{end}})$ of trip i between indices σ^{start} and σ^{end} is defined as the coherent subsequence of nodes:

$$\gamma = (i, \sigma^{\text{start}}, \sigma^{\text{end}}) \coloneqq (d_{\sigma^{\text{start}}}^i, \ldots, a_\sigma^i, d_\sigma^i, \ldots, a_{\sigma^{\text{end}}}^i),$$

where $0 \leq \sigma^{\text{start}} < \sigma^{\text{end}} \leq |i|$. The departure node $d_{\sigma^{\text{start}}}^i$ is referred to as the *hopon* of γ, and the arrival node $a_{\sigma^{\text{end}}}^i$ is referred to as the *hopoff* of γ.

Definition 3.10. An origin-destination path (OD path) P connects a starting zone z to a destination zone z' through their artificial node counterparts. Formally, an OD path P is decomposed into an initial artificial start node, a sequence of m trip segments, and a final artificial end node, represented by the concatenation:

$$P = d_z^{\text{art}} \circ \gamma_1 \circ \gamma_2 \circ \ldots \circ \gamma_m \circ a_{z'}^{\text{art}},$$

where each $\gamma_k = (i_k, \sigma_k^{\text{start}}, \sigma_k^{\text{end}})$ is a valid segment of a trip $i_k \in \mathcal{I}$. Let d_k denote the hopon event of segment γ_k and a_k its hopoff event. This path concatenation is structurally valid within our graph $G(\epsilon, \beta, h)$ if and only if its concatenation arcs are included in $\mathcal{A}$:

$$\begin{aligned}
&(d_z^{\text{art}}, d_1) \in \mathcal{A}_{\text{board}}(h), \\
&(a_m, a_{z'}^{\text{art}}) \in \mathcal{A}_{\text{alight}}, \\
&\left(a_k, d_{k+1}\right) \in \mathcal{A}_{\text{trans}}(\epsilon, \beta) \quad \forall\, 1 \leq k < m.
\end{aligned}$$

Let $o = d_1$ be the immediate departure node succeeding d_z^{art}, (we also say P *emanates* in o) and let $a = a_m$ be the final arrival node preceding $a_{z'}^{\text{art}}$. Following the general path weight definition and the telescoping properties established in Remark 1, the total travel time $w(P)$ of the journey can be computed highly efficiently. Instead of aggregating all individual travel and transfer arcs, the total path weight simplifies to:

$$w(P) = \mathsf{walk}(z, o) + \tau\left(a\right) - \tau\left(o\right) + \mathsf{walk}(a, z') = \tau\left(a, z'\right) - \tau\left(z, o\right).$$

This formulation heavily exploits the time-expanded graph structure, as it relies strictly on the boundary walking constraints (walk) and the temporal difference of the core railway sequence, minimizing arithmetic tracking operations during path exploration.

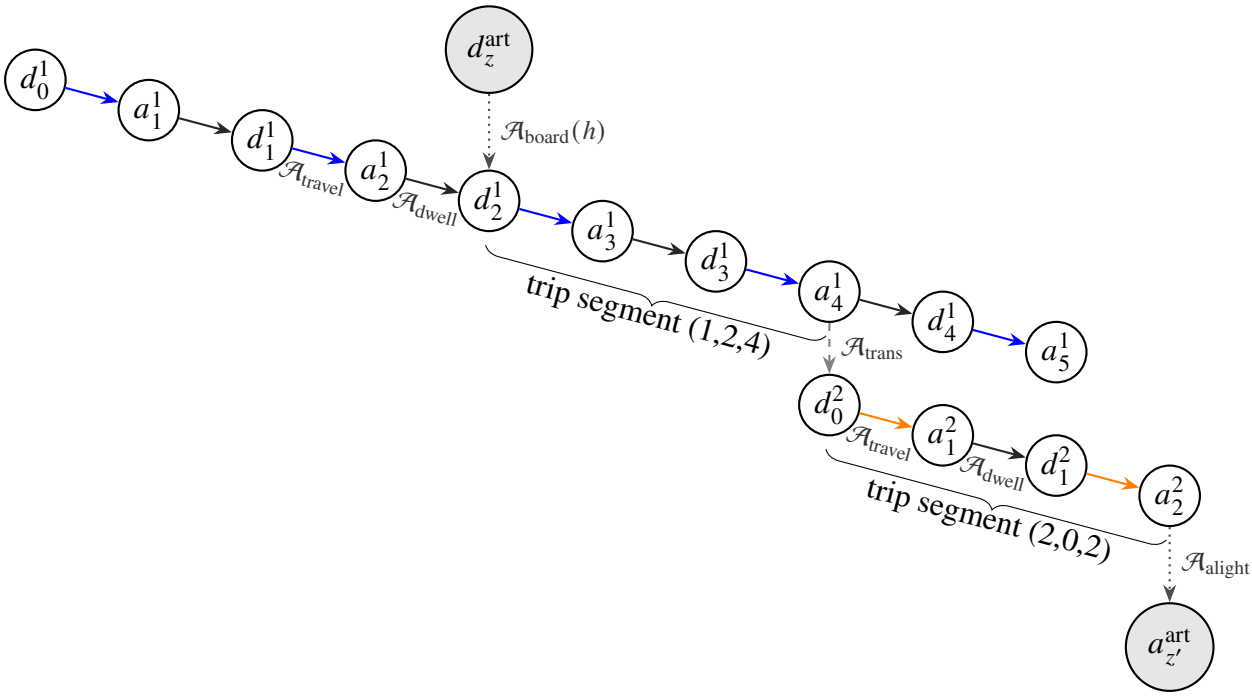


Figure 3.1: Example of an OD path $P = d_z^{\text{art}} \circ \gamma_1 \circ \gamma_2 \circ a_{z'}^{\text{art}}$ connecting zone z to z', initiated during time period h. The path utilizes a boarding arc to enter the blue trip 1 at index 2. Following the segment $\gamma_1 = (1, 2, 4)$, a gray transfer arc connects to the first departure node of the orange trip 2. It proceeds along $\gamma_2 = (2, 0, 2)$ until taking an alighting arc to reach the destination zone z'. Labels indicate the specific edge sets.

4 Problem Definitions and Algorithmic Search Spaces

Having established the structural foundation of our graph, we must now transition from purely defining elements to evaluating journeys within this network. We establish a rigorous framework for comparing paths.

To this end, we first establish routing objectives by defining the exact optimization problems to be solved. Subsequently, we introduce the core algorithmic challenges that arise when waiting times are bounded. While classic shortest-path formulations heavily rely on the assumption of sub-path optimality [5, Chapter 22], we mathematically analyze how Bellman's principle does not apply when restricting the maximum waiting time ($\beta < \infty$). This structural failure fundamentally alters the explorable path domain. To address this, we formalize a hierarchy of algorithmic search spaces and define the concept of locally stepwise earliest paths. This is necessary due to a greedy pruning strategy that trades global exactness for faster query performance and realistic passenger emulation, culminating in a specialized variant of the All-Pairs Shortest Path problem.

4.1 Dominance and APSP

Because of the time-expanded structure of our event nodes, a simple node-based path comparison does not suffice (all paths between the same nodes in $\mathcal{N}_{\text{trip}}$ share the same path weight as shown in Remark 1). Therefore, we introduce a location-aware dominance relation that evaluates alternative paths based on their station- or zone-affiliation, serving as the main criterion for discarding suboptimal journeys during our round-based exploration.

Definition 4.1 (Station- and Zone-Based Dominance)**.** Let P_1 and P_2 be two valid paths in G. Let u_1 and v_1 denote the initial and terminal nodes of P_1, and let u_2 and v_2 denote the initial and terminal nodes of P_2, respectively. We say that P_1 is *dominated* by P_2, denoted as $P_2 \prec P_1$ if they originate from the same start station or zone and terminate at the same end station or zone, but P_2 achieves a strictly lower travel time. Formally, this relation requires that $w(P_2) < w(P_1)$ and one of the following structural cases holds:

1. **Pure Station-to-Station Match** (for $u_1, v_1, u_2, v_2 \in \mathcal{N}_{\text{trip}}$):
   The paths operate between the same physical locations:

$$\mathsf{st}(u_1) = \mathsf{st}(u_2) \quad \text{and} \quad \mathsf{st}(v_1) = \mathsf{st}(v_2).$$

2. **Artificial Start to Station Match** (for $u_1, u_2 \in \mathcal{N}_{\text{start}}$ and $v_1, v_2 \in \mathcal{N}_{\text{trip}}$):
   Both paths originate from the exact same start zone z and terminate at the same station:

$$u_1 = u_2 = d_z^{\text{art}} \quad \text{and} \quad \mathsf{st}(v_1) = \mathsf{st}(v_2).$$

3. **Station Match to Artificial End** (for $u_1, u_2 \in \mathcal{N}_{\text{trip}}$ and $v_1, v_2 \in \mathcal{N}_{\text{end}}$):
   Both paths originate from the same station and terminate at the same end zone z':

$$\mathsf{st}(u_1) = \mathsf{st}(u_2) \quad \text{and} \quad v_1 = v_2 = a_{z'}^{\text{art}}.$$

4. **Artificial Origin to Artificial End Match** (for $u_1, u_2 \in \mathcal{N}_{\text{start}}$ and $v_1, v_2 \in \mathcal{N}_{\text{end}}$):
   Both paths represent full journeys connecting the same starting and end zones:

$$u_1 = u_2 = d_z^{\text{art}} \quad \text{and} \quad v_1 = v_2 = a_{z'}^{\text{art}}.$$

A path P_2 is called *optimal* or *non-dominated* within a given set of paths if there exists no alternative path P_1 within that set such that $P_1 \prec P_2$.

With this definition of dominance we can define our specification of an All-Pairs Shortest Path Problem:

Definition 4.2 (All-Pairs Shortest Path Problem on Zones)**.** Given a graph $G = (\mathcal{N}, \mathcal{A})$ and a family of zones $\mathcal{Z}$ defined over the stations of G, the *All-Pairs Shortest Path (APSP) problem on Zones* requires finding a valid non-dominated path P from d_z^{art} to $a_{z'}^{\text{art}}$ for every OD pair of zones $(z, z') \in \mathcal{Z} \times \mathcal{Z}$.

In our approach to solve the APSP problem, we will solve the following problem:

Definition 4.3 (Single Source Shortest Path Problem on Stations)**.** Given a graph $G = (\mathcal{N}, \mathcal{A})$, let $u \in \mathcal{N}_{\text{trip}}$ be a specific root node, the *Single Source Shortest Path (SSSP) problem* on stations aims to find a subgraph $T \subset G$ that forms a tree rooted at u. For every station $s \in \mathcal{S}$, the tree T must contain one non-dominated valid path P (if there is any) from u to node $v \in \mathcal{N}$ such that $\mathsf{st}(v) = s$.

4.2 Search Space

To avoid a time-consuming brute-force search over all possible valid paths, we will employ a technique to discard suboptimal paths early in the process. We discuss this technique and its influence on the search space consisting of all paths that can possibly be found within our restricted graph $G(\epsilon, \infty, h)$.

Definition 4.4 (Zone to Zone Search Space)**.** Let $\beta \in \mathbb{R}^+ \cup \{\infty\}$ be the maximum permissible transfer waiting time. We formally define the *(Zone to Zone-) Search Space* $X(\beta)$ as the set of all valid paths in $G(\epsilon, \beta, h)$ connecting any origin zone $z \in \mathcal{Z}$ to any destination zone $z' \in \mathcal{Z}$:

$$X(\beta) := \bigcup_{(z,z') \in \mathcal{Z} \times \mathcal{Z}} \left\{ P = (d_z^{\text{art}}, \ldots, a_{z'}^{\text{art}}) \mid P \text{ is a valid path in } G \right\}.$$

Therefore $X(\infty)$ represents all paths in $G(\epsilon, \infty, h)$, i.e., there is no upper bound on the transfer time. SERPENT does not explore $X(\beta)$ to the full extend. In the next Section 4.3 we will explain its limited search space $Y(\beta) \subseteq X(\beta)$ that will be defined later in Section 4.3. For now, we just introduce the starred superscript notations:

$$X^*(\beta) \subseteq X(\beta),$$
$$Y^*(\beta) \subseteq Y(\beta),$$

to denote all non-dominated paths within the respective search space. We also introduce the subscript notation for a valid sub-path P from u to v:

$$X_P(\beta),$$
$$Y_P(\beta),$$

to denote the explorable path tree, that follows after P. This path tree is rooted in the end node v of path P and is contained within the respective search space $X(\beta)$ or $Y(\beta)$.

4.3 Sub-path Optimality and Algorithmic Restrictions

To understand how these search spaces deviate from each other, we recall the concept of *sub-path optimality*, also known as *Bellman optimality*. However, we first must establish the following structural property regarding path exploration in our graph:

Theorem 4.3.1 (Monotonicity of Explorable Paths for Unrestricted Transfer Times)**.** *Let P_1 and P_2 be valid sub-paths starting from the same origin zone $z \in \mathcal{Z}$ and arriving at the same*

intermediate station $s = \mathsf{st}(a_1) = \mathsf{st}(a_2)$ *at arrival nodes* $a_1 \in P_1$ *and* $a_2 \in P_2$*, respectively. If* P_1 *arrives earlier than or at the same time as* P_2*, i.e.,*

$$\tau(a_1) \leq \tau(a_2),$$

then for $\beta = \infty$ *and any* $\epsilon \geq 0$*, the set of explorable subsequent paths is monotonically increasing with respect to an earlier arrival time:*

$$X_{P_2}(\epsilon, \infty) \subseteq X_{P_1}(\epsilon, \infty).$$

Proof. To show that $X_{P_2}(\epsilon, \infty) \subseteq X_{P_1}(\epsilon, \infty)$, we must prove that any valid path extension of P_2 is also a valid continuation for P_1. Let $d \in \mathcal{N}$ be the hopon departure node of an arbitrary valid subsequent trip segment following station s in P_2. Then transfer arc $(a_2, d) \in \mathcal{A}_{\text{trans}}(\infty, \epsilon)$.

Thus, according to Inequation 3.3 for relaxed transfer arcs, the time window constraint for a_2 connecting to d is satisfied:

$$\mathsf{walk}(a_2, d) - 2\epsilon \leq \tau(d) - \tau(a_2) \leq \infty + 2\epsilon.$$

Since $\beta = \infty$, the upper bound condition is trivially satisfied for any arrival node. Now, consider the earlier arrival node a_1 with $\tau(a_1) \leq \tau(a_2)$. Substituting $\tau(a_1)$ into the time difference yields:

$$\tau(d) - \tau(a_1) \geq \tau(d) - \tau(a_2) \geq \mathsf{walk}(a_2, d) - 2\epsilon.$$

Since both nodes arrive at the same station, $\mathsf{walk}(a_1, d) = \mathsf{walk}(a_2, d)$, the lower bound constraint for a_1 is not violated either.

Consequently, $(a_2, d) \in \mathcal{A}_{\text{trans}}(\infty, \epsilon)$ implies $(a_1, d) \in \mathcal{A}_{\text{trans}}(\infty, \epsilon)$. Hence, any valid extension in $X_{P_2}(\epsilon, \infty)$ is structurally feasible for P_1 as well, implying $X_{P_2}(\epsilon, \infty) \subseteq X_{P_1}(\epsilon, \infty)$. □

The following theorem is a deviant of the fundamental Bellman principle, that states any sub-path of an optimal path is optimal itself [4], [18, Lemma 4]).

Theorem 4.3.2 (Bellman Optimality for $\beta = \infty$)**.** *Let* $P^* \in X^*(\epsilon, \infty)$ *be an optimal o–v-path (without restrictions to waiting times) for some nodes* $o, v \in \mathcal{N}$*. Then, for every intermediate departure node* $d \in P^*$*, there is no* d'*–*v'*-path* P' *with the following properties:*

1. ***between same stations (SS):*** $\mathsf{st}(d) = \mathsf{st}(d')$ *and* $\mathsf{st}(v) = \mathsf{st}(v')$,
2. ***later departure (LD):*** $\tau(d') \geq \tau(d)$,
3. ***earlier arrival(EA):*** $\tau(v') < \tau(v)$.

Proof. Let $o, v \in \mathcal{N}$ be some start node and some target node. Let $P^* \in X^*(\epsilon, \infty)$ be an optimal o–v-path:

$$P^* = (o, \ldots, a, d, \ldots, v) .$$

Assume for contradiction that for station $s = \mathsf{st}(a) = \mathsf{st}(d)$, there is a d'–v'-path $EP' =$ fulfilling these three properties(SS, LD, EA). Then we can concatenate the starting segments of P^* with EP':

$$P' = (o, \ldots, a) \circ (d', \ldots, v') .$$

Since P^* is valid, $\beta = \infty$ and $\tau(d') \geq \tau(d)$ (LD), the concatenation in s is valid as well, $(a, d') \in \mathcal{A}_{\text{trans}}(\epsilon, \infty)$. Both paths start in o and P' arrives at the same station as P^* (SS), but an earlier arrival time (EA), thus P' has a shorter travel time than P^*:

$$w(P') < w(P^*) .$$

The existence of P' contradicts the optimality of P^*.

□

Remark 2. *This is also known as a* non-overtaking *or first-in-first-out* FIFO *property, describing that "later departures cannot lead to earlier arrivals" [2]. For our structure we should rephrase that to* later departures cannot lead to additional path exploration. *Remember that this Lemma requires unrestricted waiting times. For restricted maximum waiting time β, the set of feasible subsequent transfer arcs from arrival nodes does not scale monotonically with an earlier arrival time, i.e. later departures* can *explore additional paths and lead to earlier arrivals.*

One basic intention of our thesis is the optimization of transfer times. This is why our input data contains transfers limited by a maximum waiting time β, but this causes a major issue: The sub-path optimality does not hold anymore: Bellman holds for unconstrained single criterion problems with a monotone increasing criterion, e.g., for a shortest path with minimum total weight for non-negative arc weights. For labeling-setting algorithms in general, the Bellman sub-path optimality does not necessarily hold. For the price-optimal earliest arrival problem in public transit, the same issue for fare zones is mentioned by Euler, Lindner, and Borndörfer [11].

Theorem 4.3.3 (Correctness of Station-Based Greedy Pruning for $\beta = \infty$)**.** *For unrestricted waiting times ($\beta = \infty$) and any shift parameter $\epsilon \geq 0$, a station-based greedy pruning strategy is correct. More formally, let P_1, P_2 be o–s sub-paths of explorable paths in $X(\beta)$ with P_1 arriving at an intermediate station s with an earlier or equal arrival time than P_2, i.e.,*

$$P_1 = (o, \ldots, a_1) ,$$

$$P_2 = (o, \ldots, a_2)\,,$$

$$\tau(a_1) \leq \tau(a_2) \quad \textit{where } a_1 \in P_1,\ a_2 \in P_2,$$

then P_2 cannot be a sub-path of any globally optimal path $P^* \in X^*(\epsilon, \infty)$ and can safely be pruned.

Proof. We prove the theorem by showing that any valid continuation of the pruned path P_2 can be replaced by an alternative continuation of the path P_1 without violating any constraints.

Let P_1 and P_2 be valid o–s-paths emanating from o to an intermediate station s, arriving at nodes $a_1 \in P_1$ and $a_2 \in P_2$ respectively, such that $\tau(a_1) \leq \tau(a_2)$. Assume for contradiction that P_2 is a sub-path of a globally optimal path $P^* = P_2 \circ EP^*$ with $P^* \in X^*(\epsilon, \infty)$, but $EP^* \in X_{P_2}(\epsilon, \infty) \setminus X_{P_1}(\epsilon, \infty)$, i.e., no continuation of P_1 can explore EP^*. Here, EP^* denotes the remaining path segment sequence to any final destination zone z'.

Let $d \in \mathcal{N}$ with $\mathsf{st}(d) = s$ be the immediate subsequent departure node in EP^*. Since P^* is a valid path in $G(\epsilon, \infty, h)$, the transfer arc (a_2, d) must be valid, meaning $(a_2, d) \in \mathcal{A}_{\text{trans}}(\infty, \epsilon)$.

By Theorem 4.3.1, the set of explorable subsequent paths is monotonically increasing with an earlier arrival time when $\beta = \infty$. Thus, $\tau(a_1) \leq \tau(a_2)$ implies that $X_{P_2}(\epsilon, \infty) \subseteq X_{P_1}(\epsilon, \infty)$. Consequently, the transfer arc (a_1, d) is guaranteed to be valid as well, and EP^* can be utilized as a feasible continuation for P_1.

We can therefore construct the alternative composite path $P = P_1 \circ EP^*$. Since both paths are structurally valid, emanate at the exact same node o, and terminate with the identical segment EP^*, their total travel times are identical:

$$w(P) = w(P^*)\,.$$

This directly contradicts our initial assumption that $EP^* \notin X_{P_1}(\epsilon, \infty)$, as we have shown that a valid alternative path P with identical travel time can be explored through P_1. Hence, discarding P_2 is correct and does not reduce the reachable solution space of non-dominated paths. □

Lemma 4.3.4 (Failure of Station-Based Greedy Pruning for $\beta \neq \infty$)**.** *For a restricted maximum waiting time $\beta < \infty$, the station-based greedy pruning strategy established in Theorem 4.3.3 is no longer correct. Discarding a dominated intermediate sub-path can prevent a greedily pruning algorithm from finding a globally optimal solution.*

Proof. We prove this by showing that the monotonicity property from Theorem 4.3.1 breaks down for $\beta < \infty$, which invalidates the proof of Theorem 4.3.3.

Consider two sub-paths where P_1 arrives at node a_1 and P_2 arrives at node a_2 at station s with $\tau(a_1) < \tau(a_2)$. The greedy strategy retains P_1 and prunes P_2 because P_1 arrives earlier and thus dominates P_2 locally at station s.

Now, let d be a subsequent departure node at station s tightly scheduled after a_2 such that the transfer window satisfies:

$$\tau(d) - \tau(a_2) = \beta + 2\epsilon.$$

This transfer is valid for the slower sub-path, meaning $(a_2, d) \in \mathcal{A}_{\text{trans}}(\epsilon, \beta)$. However, for the earlier arrival a_1, the waiting time increases, yielding:

$$\tau(d) - \tau(a_1) > \beta + 2\epsilon.$$

Consequently, $(a_1, d) \notin \mathcal{A}_{\text{trans}}(\epsilon, \beta)$ because the maximum waiting time constraint is violated.

Thus, the set of explorable subsequent paths does not scale monotonically, i.e., $X_{P_2}(\epsilon, \beta) \not\subseteq X_{P_1}(\epsilon, \beta)$. If EP^* denotes a valid remaining path segment sequence to the destination starting at departure d, this continuation is only explorable via the locally suboptimal sub-path P_2. Since the greedy strategy pruned P_2, the algorithm fails to discover the globally optimal path $P^* = P_2 \circ EP^*$. □

The structural failure of Bellman's principle for restricted waiting times ($\beta < \infty$) implies that a standard Pareto- or arrival-time-bound dominance pruning on station-level is not exact, but rather heuristic. SERPENT utilizes a round-based greedy pruning strategy (for each emanation node o retaining only the earliest arrival at a station per transfer round) to greatly reduce memory usage and execution time. It does not explore the entire valid path space $X(\beta)$. We will quantify the exploration loss in Chapter 6. To formally characterize the restricted search space $Y(\beta)$ generated by this algorithmic design choice, we must formally define the structural properties of the paths it explores. This leads to the concept of *locally stepwise earliest* paths, which models the greedy, round-by-round inductive path construction:

Definition 4.5 (Locally Stepwise Earliest Path)**.** Given a graph $G = (\mathcal{N}, \mathcal{A})$ and a fixed start departure node $o \in \mathcal{N}$, a valid path P from o to a node $v \in \mathcal{N}$ with $r \geq 0$ transfers (consisting of $r + 1$ trip segments) is called *locally stepwise earliest* (SE) if it is constructed inductively as follows:

1. **Base Case ($r = 0$):** The path P consists of a single trip segment γ within the trip i originating at o. Let $s = \mathsf{st}(v)$. This path is SE if v is the earliest possible arrival node at station s reachable from o without any transfers. We denote this arrival node as $EA_0(o, s) \coloneqq v$.

2. **Inductive Step ($r > 0$):** The path P to station $s = \mathsf{st}(v)$ is formed by extending a valid SE path P' with $r - 1$ transfers from o to u with station $s' = \mathsf{st}(u)$. P' is extended by a final trip segment $\gamma = (i, \sigma^{start}, \sigma^v)$ to arrival node $v = a^i_{\sigma^v}$.

   This path is SE if and only if:

- **Valid transfer:** The transfer at s' from the arrival of the previous SE path to the new segment is valid:
$$\left(EA_{r-1}(o, s'),\ d^{i}_{\sigma^{start}}\right) \in \mathcal{A}_{\text{trans}}(\epsilon, \beta),$$
- **Earliest:** Its arrival time $\tau(v)$ at s is optimal for r transfers, meaning v minimizes the arrival time among all such valid single-segment extensions from any SE path with $r-1$ transfers. We denote this local minimum as $EA_r(o, s)$.
- **Strict Improvement / Dominance:** Its arrival time $\tau(v)$ at s is strictly better than all SE path with fewer transfers:
$$\tau\left(EA_r(o, s)\right) < \tau\left(EA_{r-1}(o, s)\right)$$

Now we can define $Y(\beta)$ similar to definition 4.4.

Definition 4.6 (Stepwise Earliest Search Space)**.** Let $\beta \in \mathbb{R}^+ \cup \{\infty\}$ be the maximum permissible transfer waiting time. We define the *(Zone to Zone- Locally) Stepwise Earliest Search Space* $Y(\beta)$ as the set of all valid locally stepwise earliest paths in $G(\epsilon, \beta, h)$ connecting any origin zone $z \in \mathcal{Z}$ to any destination zone $z' \in \mathcal{Z}$. Formally,

$$Y(\beta) := \bigcup_{(z,z') \in \mathcal{Z} \times \mathcal{Z}} \left\{ P = (d^{\text{art}}_{z}, \ldots, a^{\text{art}}_{z'}) \mid P \text{ is a valid locally stepwise earliest path in } G \right\}.$$

Remark 3. *We observe that the search framework built by locally stepwise earliest paths aligns closely with the natural heuristic behavior of a typical passenger. When planning a journey under bounded transfer times, a passenger naturally seeks to minimize the arrival time at any intermediate transfer station. Instead of evaluating mathematically complex multi-criteria tradeoffs across the entire global network, passengers inherently act as greedy decision-makers at each transfer node, choosing the earliest available connection that satisfies their comfort constraints. Therefore, the restricted search space $Y(\beta)$ practically mirrors realistic routing choices.*

Consequently, the final output delivered by SERPENT is not the global exact solution to the unconstrained APSP problem, but rather the exact solution within the restricted boundaries of locally stepwise earliest arrival pathfinding. This restriction guarantees rapid query times and mirrors realistic passenger behavior, while bounding the optimization domain to paths that can be discovered via greedy station-based pruning. To clearly delineate the exact theoretical objective from our algorithmic output, we formally define this target solution as follows:

Definition 4.7 (Stepwise Earliest All-Pairs Shortest Path Problem on Zones)**.** Given a graph $G = (\mathcal{N}, \mathcal{A})$ and a family of zones $\mathcal{Z}$ defined over the stations of G, the *Stepwise Earliest*

All-Pairs Shortest Path (SEAPSP) problem on Zones requires finding a valid non-dominated path P from d_z^{art} to $a_{z'}^{\text{art}}$ for every OD pair of zones $(z, z') \in \mathcal{Z} \times \mathcal{Z}$ *within the restricted search space of locally stepwise earliest paths*.

By establishing the SEAPSP definition, we emphasize that SERPENT remains mathematically exact with respect to its designated search domain $Y(\beta)$, while the structural loss of global paths $(X^*(\epsilon, \beta) \setminus Y^*(\epsilon, \beta))$ is classified as a network-inherent property that we will analyze and quantify in Chapter 6.

Remark 4. *Consequently, the structural differences and pruning mechanisms restrict the exploration domain. Hence the search spaces in general form a strict hierarchical inclusion:*

$$Y(\beta) \subset X(\beta) \subset X(\infty).$$

This relation visualizes why the greedy algorithm operates on a restricted solution space, where the non-exploration of certain paths is the direct consequence of the maximum waiting time constraint β.

5 Algorithm

We designed our algorithm SERPENT to utilize the following components as much as possible:

1. the structure and rich content of our input data (explained in Section 6.1), especially the pre-calculated list of possible node-to-node transfers,

2. advantages of the algorithmic search style of the RAPTOR algorithm if applied in public transit networks [6] .

In order to yield a very fast calculation, for each zone z in $\mathcal{Z}$, the first version of the algorithm only considered the earliest arrival time at each station reachable from start node d_z^{art} as a path comparison criterion. This basically solves the SSSP (see definition 4.3) for root node d_z^{art}. But then, any path P departing early within time period h from the starting zone z is naturally favoured, regardless of the travel time. Following a path that starts 30 minutes later than P within the same period h might arrive at the terminal station 15 minutes later than P, yielding a better travel time. Noticing that we can find all possible departures $\delta^+(d_z^{\text{art}})$ reachable from zone z in our input data efficiently, we upgraded the algorithm to optimize travel time instead of earliest arrival time. Effectively, this forced us to solve the SSSP not only for root d_z^{art}, but for each root node $o \in \delta^+(d_z^{\text{art}})$, greatly increasing computational effort. Since the focus of SERPENT was to terminate within seconds, pruning methods were inevitable.

5.1 RAPTOR Algorithm

The Round-Based Public Transit Routing (RAPTOR) algorithm, introduced by Delling, Pajor, and Werneck [6], is an exact router designed specifically for public transit networks. It solves the SSSP in $O\big(R \cdot |\mathcal{N}|\big)$, where R is the maximum number of transfer-rounds. Unlike traditional Dijkstra-based approaches, RAPTOR does not need to maintain a priority queue and operates entirely without graph edges. Instead, it exploits the linear structure of transit routes and processes the network in discrete rounds.

Each round r computes the earliest arrival times at all stops using at most $r - 1$ transfers. This round-based decomposition inherently provides a Pareto-optimal set of journeys balancing arrival time against the number of transfers.

Starting at a certain timestamp τ_{dep} the algorithm maintains an array of earliest arrival times $\tau_r(s)$ (similar to our $EA_r(o, s)$ for start node o) for each station s and round r. During a round, it only inspects routes that pass through stations that were updated (marked) in the previous round. For each such route, it scans stations sequentially, tracking the earliest possible trip that can be boarded based on the arrival times from round $r - 1$. After processing the routes, foot-transfers are evaluated to finalize the round. The algorithm terminates when a round completes without marking any new stations.

Algorithm 5.1 outlines the standard single-source RAPTOR procedure based on the original framework.

Input: Timetable containing routes $\mathcal{R}$ and stations $\mathcal{S}$, source station s_{start}, departure time τ_{dep}
Output: Earliest arrival times $\tau_r(s)$ for all stations $s \in \mathcal{S}$ and rounds r

```
// Initialization step
for each round r and station s ∈ S do
    τ_r(s) ← ∞
τ_0(s_start) ← τ_dep
MarkedStations M ← {s_start}
r ← 1
// Main Loop: Iterate over rounds
while M ≠ ∅ do
    // Step 1: Determine routes serving marked stations as pairs (r, s) ∈ Q
    Queue Q ← ∅                          // Accumulate pairs of (route, station)
    for station s ∈ M do
        for each route r ∈ R passing through s do
            if there is no (r, s') ∈ Q then
                Q ← Q ∪ {(r, s)}
            else if s appears earlier in r than s' then
                Q ← (Q \ {(r, s')}) ∪ {(r, s)}      // Keep only the earliest station
    M ← ∅                                // Clear marked stations for this round
    // Step 2: Process each accumulated route-station pair
    for each pair (r, s_marked) ∈ Q do
        Trip t ← null
        for each station s along route r starting from s_marked do
            if t ≠ null then
                τ_arr ← arrival time of trip t at station s
                if τ_arr < τ_r(s) then
                    τ_r(s) ← τ_arr
                    M ← M ∪ {s}                  // Mark station for next round
            // Check if we can catch an earlier trip of route r
            if τ_{r-1}(s) < departure time of trip t at station s (or t = null) then
                t ← earliest trip in r departing after τ_{r-1}(s)
    // Step 3: Process footpaths / transfers
    for each station s ∈ M do
        for each transfer station s' reachable from s via footpath with duration walk do
            if τ_r(s) + walk < τ_r(s') then
                τ_r(s') ← τ_r(s) + walk
                M ← M ∪ {s'}
    r ← r + 1
```

Figure 5.1: The Standard Single-Source RAPTOR Algorithm

5.2 SERPENT Algorithm

Our algorithm 5.2 is a *Stepwise Earliest Raptor-like Path Explorer for Nodes that are Trip-induced* (SERPENT). For a reminder on the method of stepwise earliest, see definition 4.5. It operates on a graph G, starting from each artificial start node in $\mathcal{N}_{\text{start}}$ and each period h. The set of all $o \in \delta^+(d_z^{\text{art}})$ is called O. The goal is for each starting zone z and end zone z' to find an optimal z–z' path amongst all locally stepwise earliest paths emanating from $o \in O$. In line 4 we need to order all emanation departures in O that can be reached from zone z by their time of starting the journey $\tau(z, o)$. This ordering is mandatory to utilize our pruning method in line 17, where we omit paths with longer travel times. We keep track of o_{prev} simply to initiate an upper bound on the arrival times in line 11. Due to our ordering of O, the walk to emanation node o will start earlier than the walk to o_{prev}. We omit any path with a longer travel time to an intermediate station s than the previously best one. This has been done similarly by Delling, Pajor, and Werneck [6] in the rRAPTOR variant of their algorithm. This variant solves a *Profile Search*, also called *Range Query*, instead of running independent individual searches. Similar to RAPTOR, for every emanation node o we store the arrival node $EA(o, s)$ that arrives fastest at station s within the currently explored path tree from o (see line 18).

$H(r, s, o)$ is the label container used to keep track of the path exploration tree. It holds information about the last trip segment γ_r of path P, emanating from o. P arrives at station s from node o after the r-th transfer. $H(r, s, o)$ gets updated if a faster path from o to s gets explored during the same round r. Since it is important for path reconstruction, we also store a pointer to the last used trip segment (and if a shift was required for the last trip segment) in round $r - 1$. We also keep track of the stations with improvements to their earliest arrival EA in a set NS (a collection of the Next Stations). If improvements to EA have been done, NS will become the set of current stations CS during the next round r. Each round, for each station in CS we investigate all reachable transfer departures $d \in \delta^+(EA(o, s))$ in line 14. For each of these d, we scan for consecutive arrival nodes a and if they can improve their respective $EA(o, s')$ with $s' = \mathsf{st}(a)$. Note that we could easily limit r and output the best possible connections for a maximum number of r transfers early if desired.

After path trees for all emanation nodes in O have been explored, we review which end zones can be reached via $\mathcal{A}_{\text{alight}}$. Including both weights of the walks $\mathsf{walk}(z, o)$ from the starting zone z and $\mathsf{walk}(a, z')$(a being $EA(o, s)$) towards end zone z', we choose the path with lowest travel time amongst all o in line 27 and update the best possible path towards z'. Note that multiple zones might be in walkable distance of s.

Ultimately we reconstruct the path segments of the best path recursively via $H(r, s, o)$ from last transfer round r until we reach o and output the path and its segments in time-wise ascending order.

Input: Timetable $\mathcal{T}$ with nodes and arcs, zones $\mathcal{Z}$ and periods $\mathcal{H}$
Output: One path from artificial start d_z^{art} to end node $a_{z'}^{\text{art}}$ for each OD pair (z, z') and period h, optimal among all locally stepwise earliest path satisfying (z, z')

`// Parsing Step: Preprocessing of network data`
Parse list of stop events list and list of transfers

`// Main Loop: Iterative path exploration`
for *each starting zone* $z \in \mathcal{Z}$ *and starting period* $h \in \mathcal{H}$ **do**
  Get start departures $O = \delta^+(d_z^{\text{art}})$, starting within h
  sort $o \in O$ by descending $\tau(z, o) \in h$
  $o_{\text{prev}} \leftarrow$ null, $\forall s \in \mathcal{S} : \tau\big(EA(null, s)\big) \leftarrow \infty$ `// initialize for first` o
  $\forall o \in O, \forall s \in \mathcal{S} : EA(o, s) \leftarrow \infty$
  for *emanation node* $o \in O$ **do**
    $r \leftarrow 0$ `// number of transfers`
    $CS \leftarrow \{\text{st}(o)\}$ `// set of current stations`
    $NS \leftarrow \emptyset$ `// stations processed during next round`
    $\forall\, s \in \mathcal{S} : \tau\big(EA(o, s)\big) \leftarrow \tau\big(EA(o_{\text{prev}}, s)\big)$ `// earliest arrival time`
    while $CS \neq \emptyset$ **do**
      `// explore paths with` r `transfers`
      for *station* s *in* CS **do**
        for *each outgoing departure* $d = d^i_{\sigma^{start}} \in \delta^+(EA(o, s))$ **do**
          for *subsequent arrival in* i*:* $a = a^i_\sigma$ *with* $\sigma > \sigma^{start}$ **do**
            $s' = \text{st}(a)$
            if $\tau(a) < \tau(EA(o, s'))\ \wedge$ *shift is valid* **then**
              $EA(o, s') \leftarrow a$ `// update earliest arrival`
              $H(r, s', o) \leftarrow (i, \sigma^{start}, \sigma)$ `// save used trip segment`
              $NS \leftarrow NS \cup \{s'\}$ `// Mark stop` s' `for next round`
      `// enable one more transfer`
      round $r \leftarrow r + 1$
      $CS \leftarrow NS$
      $NS \leftarrow \emptyset$
    `// there were no more improvements for` o `and` CS `is empty`
    $o_{\text{prev}} \leftarrow o$
  `// we are done solving locally stepwise earliest arrival problem for each` $o \in O$
  for $s \in \mathcal{S}$ **do**
    for *each end zone* $z' \in \mathcal{Z}$ *that can be walked to from* s **do**
      choose o amongst all emanation nodes O with the lowest travel time $\tau\big(EA(o, s), z'\big) - \tau(z, o)$
      update best path BP if travel time is shorter than the previous best
  reconstruct and output best path BP from d_z^{art} to all $a_{z'}^{\text{art}}$

Figure 5.2: Stepwise Earliest Raptor-like Path Explorer for Nodes that are Trip-induced (SERPENT)

Remark 5. *For $\epsilon = 0$ all of our arc weights are non-negative and all further exploration of paths require an improvement over the previous best arrival time at a station. Thus we assure that there are no loops in explored paths and SERPENT terminates. While exploring path P for $\epsilon > 0$ we might use transfer arcs (a, d) with $\tau(a) < \tau(d)$. These arcs become feasible if the trip of a gets shifted to the left and the trip of d to the right. Thus while considering transfers, SERPENT needs to keep track of the necessary shift to make the transfer in time. During path exploration, we relay the necessary shift information forward. If an ingoing arc* towards *trip i required shifting i to the right and a follow up outgoing transfer would require i to be shifted left, we omit the outgoing transfer option. In this way we prevent SERPENT to consider a path i shifted forward and backward at the same time within the same path exploration and ensure $\tau(v)$ to increase monotonously along the explored nodes v of P.*

5.3 Pruning and Limitations

To achieve computational feasibility and accelerate the pathfinding process, SERPENT refines the dominance-based pruning with a self-pruning technique.

The concept of our *dominance-based pruning* is already described in definition 4.1, while the consequences are proved in Lemma 4.3.4. SERPENT discards suboptimal paths arriving at intermediate locations by evaluating node-relative dominance on a station-level. Specifically, when both paths P, P' emanate from the same departure o we prune P at station s if it does not arrive earlier than P'. Under unrestricted waiting times, since the arrival time of P is greater than or equal to that of P', retaining P cannot yield any additional optimal connections according to Theorem 4.3.3. For the restricted case with $\beta \neq \infty$ this method of pruning will yield suboptimal path exploration, as shown in Lemma 4.3.4. Later in Chapter 6, we will quantify the loss of optimality for OD pairs using a real-world public transit network.

To accelerate our search even further, we utilize a mechanism termed *self-pruning*. It leverages sorting emanation nodes $o \in O = \delta^+(d_z^{\text{art}})$ for every zone z and execute their path exploration in descending chronological order. By processing O from the latest to earliest departure time, we can discard redundant connections efficiently. Under this paradigm, tracking or comparing full path weights and travel times becomes unnecessary. A simple, direct comparison of arrival times suffices to prune suboptimal paths because any previously processed emanation node o' is already guaranteed to have a later departure time than the current emanation node o (i.e., $\tau(o') > \tau(o)$).

The following theorem underlines that the inequality of these search spaces does not merely derive from an algorithm design error. It is a conscious choice we made to enforce the simulation of expected passengers behaviour and preference for short transfer times. The theorem basically

states that each path P not explored by SERPENT could be composed of an explored start segment P_Y and a transfer forcing an unacceptably long waiting time to a remaining sub-path of P.

Theorem 5.3.1 (Missed Paths)**.** *Let P be a valid path serving the OD pair (z, z') that exists in the full solution space but is not explored by SERPENT, i.e., $P \in X^*(\beta) \setminus Y(\beta)$. Then, there exists a station $s \in \mathcal{S}$, an arrival node a_Y at s reached by an explored sub-path P_Y, and a departure node d_X at s starting the remaining sub-path P_X of P such that P can be decomposed or matched via a concatenation at s:*

$$P' = P_Y \circ P_X,$$

where:

$$P_Y = (d_z^{art}, \ldots, a_Y) \in Y(\beta) \quad with \quad \mathsf{st}(a_Y) = s,$$
$$P_X = (d_X, \ldots, a_{z'}^{art}) \subseteq P \quad with \quad \mathsf{st}(d_X) = s,$$

and the transfer (a_Y, d_X) at station s violates the maximum waiting time constraint:

$$\tau(d_X) - \tau(a_Y) - \mathsf{walk}(a_Y, d_X) > \beta + 2\epsilon.$$

Proof. Let $P \in X^*(\beta) \setminus Y(\beta)$ be a valid path that is not fully explored by SERPENT. Since P starts at d_z^{art}, there must exist a first intermediate station $s \in \mathcal{S}$ where our algorithm deviates from P.

Let d_X be the departure node at station s that path P uses to continue its journey, and let $P_X = (d_X, \ldots, a_{z'}^{\text{art}})$ be the remaining sub-path of P from s onwards. Since s is the *first* station in P where arrival nodes do not get explored, d_X must have encountered a pruning step in s.

According to line 17, pruning takes place if and only if SERPENT has already found a path $P_Y = (d_z^{\text{art}}, \ldots, a_Y) \in Y(\beta)$ that reaches s at an arrival node a_Y with a strictly earlier or equal arrival time compared to the arrival time a_X of P at s:

$$\tau(a_Y) \leq \tau(a_X). \tag{5.1}$$

Since P is a valid path in the full solution space, the transfer occurring in P at station s to the departure d_X satisfies the waiting time bound:

$$-2\epsilon \leq \tau(d_X) - \tau(a_X) - \mathsf{walk}(a_X, d_X) \leq \beta + 2\epsilon. \tag{5.2}$$

Now, consider the alternative path concatenation $P' = P_Y \circ P_X$, which switches from the faster, explored sub-path P_Y to the remaining sub-path P_X via the transfer arc (a_Y, d_X). Since

P_Y is explored and P is valid, the only reason why SERPENT does not explore P' further, is that the transfer arc (a_Y, d_X) is invalid in our restricted network, i.e., $(a_Y, d_X) \notin \mathcal{A}_{\text{trans}}(\epsilon, \beta)$.

A transfer arc can only be invalid for two structural reasons: either the transfer is physically impossible due to insufficient buffer for the minimum transfer time, or it exceeds the allowed maximum waiting time.

Since $\mathsf{walk}(a_Y, d_X) = \mathsf{walk}(a_X, d_X)$ depends only on the station s, we can substitute inequality (5.1) in (5.2) to obtain:

$$-2\epsilon \leq \tau\big(d_X\big) - \tau\big(a_X\big) - \mathsf{walk}(a_X, d_X) \leq \tau\big(d_X\big) - \tau\big(a_Y\big) - \mathsf{walk}(a_Y, d_X).$$

This proves that the arrival a_Y occurs early enough to physically catch the departure d_X.

Consequently, the transfer arc (a_Y, d_X) can only be excluded from $\mathcal{A}_{\text{trans}}(\epsilon, \beta)$ because it violates the upper bound constraint on the waiting time, which is exactly our statement:

$$\tau\big(d_X\big) - \tau\big(a_Y\big) - \mathsf{walk}(a_Y, d_X) > \beta + 2\epsilon.$$

Thus, the omitted path P always implies the existence of a faster, explored alternative P_Y up to station s, where the transfer to the remaining route P_X would force an unacceptably long waiting time that exceeds our threshold. This completes the proof. □

Remark 6. *Theorem 5.3.1 implies a valuable structural insight for the optimization of a public transit network. The non-exploration of path P does not indicate a failure to find existing connections, but rather highlights a structural deficiency in the timetable: a passenger could travel faster if they were willing to endure an excessively long waiting time at station s. Therefore, the set of missed paths $X^*(\beta) \setminus Y^*(\beta)$ explicitly identifies critical OD pairs (z, z') where train shift-, schedule synchronization- or frequency improvements are required to increase connectivity of these zones.*

5.4 Possible Extensions

During the path exploration, SERPENT needs to track if a trip i needs to be shifted forward or backward to enable the explored path. We could output and analyze for each trip i how often a shift requirement occurs in an optimal path P in $Y^*(\beta)$ and quantify how often it needs to be shifted left or right to make P valid. On this basis, ideally weighted by passenger demand, we could fix a global shift assignment. The input data can then be refined to contain arcs that are only valid in $\mathcal{A}_{\text{trans}}$ within this assignment. If we drop the 2ϵ-tolerance between arrival and departure times, we can evaluate the effect of this global shifting. If we renew the 2ϵ-tolerance

for the new shifted timestamps, this procedure can lead to an iterative shifting improvement technique.

Furthermore, SERPENT increments on the number of transfers and a considerable amount of paths found within our $G(\epsilon, \beta, h)$ uses more transfers than most passengers would like to endure, especially when restricting transfers by a small maximum waiting time β. A limitation on the number of transfers might provide even more practical evaluation data for the given graph network, since passengers usually prefer a low number of transfers. Since SERPENT already runs in rounds associated with the number of transfers, this extension can be applied trivially.

SERPENT could also be fed with different input data. In our thesis we did not choose a specific global shift assignment (see Section 3.3). But we could very well create the transfer arc input data for a shift assignment and run SERPENT on this new timetable.

5.5 Algorithmic Advantages

Unlike Dijkstra-based algorithms, we do not need to order a full priority queue within our the main step. We only order the emanating departures for each starting zone once. The descending order is necessary for our self-pruning, this way we avoid to pursue sub-paths with longer travel time as fast as possible. Without the need to maintain a full priority queue each round, we are not limited by the widely known sorting barrier of $O(|\mathcal{A}| + |\mathcal{N}| \log |\mathcal{N}|)$ that these algorithms suffered from. But in fact, recently Duan et al. [10] found a Dijkstra based algorithm that significantly lowered the theoretic complexity to $O(|\mathcal{A}| \log^{2/3} |\mathcal{N}|)$. This is achieved by avoiding to sort all remaining stations. They just sort so called pivot stations each round. These pivots must first be identified as frontier stations that precede others in path trees. Subsequent nodes do not have to be strictly ordered. It suffices to estimate them well enough to ensure correctness. The runtime of their algorithm only achieves a practical time benefit for very large and sparse graphs, and thus might be very interesting for future studies of public transit. With our algorithmic design, we sort much smaller instances and skip the sorting barrier problem altogether.

We want to be able to alter any part of the whole timetable to study the consequences on shortest paths. Unlike labeling-based approaches, SERPENT does not require heavy preprocessing. The trips in file A can be in arbitrary order. So even adding or deleting stations, trips or routes can be handled without additional re-computation. Stop events belonging to the same trip are already pre-sorted in our data. This allows to build the structures for high cache locality easily and fast. Unlike the Connection Scan Algorithm (CSA) [8], stop events do not have to be chronologically reordered. Each run of SERPENT builds the necessary index lists from scratch and can handle any updates of the base data.

We need to run our path exploration for each source node. To our advantage the path exploration runs for each source node independently, a concept widely known as *embarrassingly parallel*. An embarrassingly parallel problem is defined as a computational task that can be separated into a large number of completely independent sub-tasks, requiring no communication or dependency between them during execution. Due to this independence, we can partition the source nodes d_z^{art} to different threads. In Section 6.2 we will quantify the benefit of a naive partition of the source nodes.

We do use label bags with multiple candidates for the best path in SERPENT, but ordering the emanation nodes enables us to skip many comparisons of label bags when exploration decisions are made. By running emanation nodes in departing order, we always ensure that the walking departure time of our current emanation node is lower than any previously added label. Therefore a single comparison for arrival time with the previous best arrival time suffices during the exploration of the path tree. We delay the travel time comparison until after the exploration has finished, effectively running single criterion problems (earliest arrival) to solve a bi-criterion problem (travel time).

Instead of collecting all routes running through marked stops each round, we make use of the data architecture of our input. During RAPTOR [6], the underlying data needs to be specifically ordered and associated with a route to determine the next exploration hopon d for each route efficiently, before they check at each subsequent departure if an earlier trip of the same route can be taken. We skip this necessity entirely while we loop through the marked stops. Instead we make use of the transfer arc list. It already includes footpaths. We map each arrival node to a list of its possible transfer arcs while parsing data in the beginning of SERPENT. Hence we will have no need to search for the first possible trip of each route, a simple $O(1)$ lookup suffices to find possible transfers for any arrival node.

The nodes of trip i are stored consecutively in our list, providing good *spatial locality* for the subsequent nodes that need to be checked without constantly wandering around the memory space. One can read more about this principle of program behaviour in Denning [7].

5.6 Time Complexity

These are the quantities we use to estimate the runtime of SERPENT:

Z	number of Zones	$\lvert\mathcal{Z}\rvert$,
H	number of distinguished time periods	in our case 24,
D	maximum number of emanation departures	$\max\{\lvert\delta^+(d_z^{\text{art}})\rvert \mid z \in \mathcal{Z}\}$,
R	maximum amount of transfer rounds	constant in practice,
S	number of stations	$\lvert\mathcal{S}\rvert$,
N	total number of stop nodes in all trips	$\lvert\mathcal{N}_{\text{trip}}\rvert$,
$T_{\max}$	maximum length of a trip	nearly constant in practice.

At the core of SERPENT, the path exploration of emanation o runs $r \leq R$ times until there are no more earliest arrival improvements in the explored path tree of o. For arbitrary graphs R can be as large as S. But real life public transit networks are designed to require few transfers, hence R can be treated like a constant, as done by Delling, Pajor, and Werneck [6]. While RAPTOR iterates over all routes each round, in SERPENT we utilize the structure of our data set and iterate over the reachable trips of all marked stations. If multiple stations visited by trip i are marked, the subsequent arrivals of trip i might get scanned multiple times. The worst case runtime for one round is therefore $\mathcal{O}(T_{\max} \cdot N)$ compared to $\mathcal{O}(N)$ for RAPTOR. We could avoid this issue by marking the already explored trip segments during current round r or interposing a step to determine the first index at which trip i visits a marked station for all $i \in \mathcal{I}$, similar to RAPTOR. Initial attempts to implement an elegant control mechanism for preventing redundant checks did not yield a reduction in execution time. However, since the current implementation processes the entire algorithm within a few seconds, further optimization was deemed a low priority. Consequently, this asymptotic improvement potential was not pursued further. It remains a subject for future work and should be implemented if stricter runtime constraints arise. The running time of a single query (for a specific o) thus lies within:

$$\mathcal{O}(R \cdot T_{\max} \cdot N).$$

For the theoretic worst case, our pruning does not take effect and sorting our sets O of emanation nodes does nothing for us. This happens if every subsequent emanation node $o \in O$ explores paths with lower arrivals at stations than previously found in every iteration. Thus we need to multiply the inner loop with $|O| \leq O_{max}$ and add the sorting step. To sort O, we need $\mathcal{O}(|O| \log |O|)$ comparisons. This leaves us with the following runtime to explore the path tree

for fixed z:

$$O\big(D \log D + D \cdot R \cdot T_{\max} \cdot N\big).$$

After the path exploration is done we identify the minimum travel time across all stations s to the target zones z'. That is why we iterate over all S stations to identify all reachable end zones z' from s, to know which destination zones can be updated. In real networks with a reasonable choice of zones, the number of zones within walkable distance ($\mathcal{A}_{\text{alight}}$, see Equation 3.5) from any given station s should be bounded by a small constant ($c \ll Z$), since we bound reachable zones with a maximum walking time(larger graphs cannot be made arbitrarily dense in a public transit scenario). We compare travel times amongst all emanating departures (up to D many) to choose the best path for OD pair (z, z'). Thus, the end zone evaluation block runs rather within:

$$O\big(c \cdot S \cdot D\big),$$

instead of:

$$O\big(Z \cdot S \cdot D\big).$$

Due to our history-pointer structure within $H(r, s, o)$ the segments of a path can easily be reconstructed within negligible time of $O\big(R\big)$. Running over all starting zones $z \in \mathcal{Z}$ and number of periods H leaves us with a total worst case run time of:

$$O\big(Z \cdot H \cdot \Big[\underbrace{D \log D}_{\text{sorting origins}} + \underbrace{D \cdot R \cdot T_{\max} \cdot N}_{\text{path exploration}} + \underbrace{c \cdot S \cdot D}_{\text{eval. end zone}} \Big]\big).$$

Since every departure node belongs to one or only a few zones and starting time periods, we can estimate for real-world transit networks that the total number of emanation departures across all zones and periods is proportional to the network size, i.e., $Z \cdot H \cdot D \simeq N$. Substituting this in our total run time yields:

$$O\big(N \cdot \Big[\log D + R \cdot T_{\max} \cdot N + c \cdot S \Big]\big),$$

which is clearly dominated by the quadratic path exploration term:

$$O\big(R \cdot T_{\max} \cdot N^2\big).$$

The core acceleration of our algorithmic approach to handle travel time instead of earliest arrival time relies on the descending ordering of the origin nodes O (line 4) and the propagation of upper bounds via o_{prev} (line 11).

By sorting the origins according to their latest possible departure time from zone z, the earliest arrival $EA(o_{\text{prev}}, s)$ from the previous iteration serves as a highly restrictive upper bound for

the current origin o. In practice, subsequent origin nodes often cannot find faster connections to intermediate stations than those already explored by previous origins. Consequently, the condition in line 17 evaluates to false frequently. The While-loop for current stations (CS) ensures that most trips do not need to be explored during round r. This causes the RAPTOR core to terminate much faster than in worst case after considerably fewer lookups for the majority of the origins in O.

In contrast to a worst-case scenario of a fully connected graph where every station connects to every destination zone, real-world public transit networks exhibit extreme sparsity. While theoretically bounded by the number of stations S, the maximum number of reasonable transfers R rarely exceeds a double digit integer.

In summary, due to the tight upper bounds carried over between sequential origin runs, the sparsity and inherent localized nature of transit geography, the runtime in practice behaves reasonably close to a single standard RAPTOR query ($O\big(R \cdot N\big)$) for every start node $n \in \mathcal{N}_{\text{trip}}$, which runs in $O\big(R \cdot N^2\big)$, making it highly viable for large-scale networks.

6 Experimental Results

To demonstrate the practical applicability and algorithmic efficiency of the proposed method, all computational experiments were deliberately executed on a standard, unoptimized consumer-grade laptop rather than a dedicated high-performance computing cluster.

The evaluation platform consisted of an Acer Swift SFX14-41G equipped with an AMD Ryzen 7 5800U processor (8 cores, 16 threads, base clock speed of 1.91 GHz) and 16 GB of RAM, running a standard Microsoft Windows 10 Home (64-bit) operating system. No hardware-specific code optimizations or GPU accelerations except for multi-threading were utilized during execution.

In addition to the Main-Step execution, the data preprocessing phase–which encompasses parsing the raw network data and initializing the internal data structures and mapping tables–requires an execution time of approximately 2 seconds, remaining independent of the variation in ϵ and β. Despite these basic computing conditions, SERPENT demonstrates remarkable efficiency. As shown in the performance metrics, the Main-Step execution time remains strictly within the magnitude of a few seconds for all tested parameter configurations, with a maximum runtime of only 23.52 seconds even under full parameter relaxation ($\epsilon = 5, \beta = 60$). This underlines that SERPENT is highly lightweight and fully viable for near-real-time applications without requiring specialized server infrastructure.

6.1 Input data

Our input data derives from algorithmic steps of Pedro Maristany de las Casas. Several implementations from his work about multi criterial shortest paths can be found in his PhD thesis [17]. File A contains all arrival and departure nodes, both artificial and standard. File B is a list containing every specification of our defined arcs $\forall\ h \in H$.

$$\begin{aligned} &\text{File A:} \quad \mathcal{N}_{\text{start}}, \mathcal{N}_{\text{end}}, \mathcal{S}, \mathcal{Z}, \mathcal{N}_{\text{trip}}, \mathcal{I} \\ &\text{File B:} \quad \mathcal{A}_{\text{board}}, \mathcal{A}_{\text{travel}}, \mathcal{A}_{\text{dwell}}, \mathcal{A}_{\text{trans}}, \mathcal{A}_{\text{alight}} \end{aligned}$$

For our input we use the real-life timetable information provided from a large European passenger railway operator. A new set of input files is provided for each combination of

$\beta \in \{15, 30, 45, 60\}$ and $\epsilon \in \{0, 1, 2, 3, 4, 5\}$. Each of these sets contains of two different input files. File A provides the timetable in form of a list. Stop events are stored aggregated by their trips. Nodes within a trip are sorted in their natural sequence, ascending by their timestamp τ, whereas the trips themselves can appear in an arbitrary order. The data is limited to all trips operating within 24 hours of a given day. Each data line contains information of a stop event, either departure or arrival and therefore relates to a node in our graph. Our preprocessing step stores a list of stop events from this file, extracting the trip-ID, station-ID and of course the timestamp τ of arrival or departure. Inducing a proper mapping early accelerates the main execution: Whenever we consider taking a trip $i \in \mathcal{I}$, we have a fast $O(1)$ lookup to get access to the hopon node and its successor nodes in i, stored in a compact consecutive order.

Each zone contains some stations usually close to each other in the geographic sense. For each of these zones z and time period $h \in [0, 23)$, file A also contains source node $d_z^{\text{art}} \in \mathcal{N}_{\text{start}}$. Our artificial end nodes $a_z^{\text{art}} \in \mathcal{N}_{\text{end}}$ follow right behind them, leaving us at $24 \times |\mathcal{Z}|$ source nodes and therefore $24 \times |\mathcal{Z}|^2$ OD triples. In our data set we use 800 zones; hence SERPENT may output up to 15.360.000 paths.

File B contains tail, head and arc weight for every specification of arcs $\mathcal{A}$. Due to the trip aggregation in File A and the path weight getting calculated by the start and end of the path (see Remark 1), $\mathcal{A}_{\text{travel}}$ and $\mathcal{A}_{\text{dwell}}$ do not matter for us. We extract from File B all arcs from artificial start to departure nodes $\mathcal{A}_{\text{board}}$, those from arrival to artificial end nodes $\mathcal{A}_{\text{alight}}$ and the (for SERPENT highly valuable) transfer arcs $\mathcal{A}_{\text{trans}}$. Since we are interested in the influence of maximum waiting times and shifting the timetable, we have separate files for each fixed maximum waiting time β and shift limit ϵ. Note that the transfer arc entries are highly dependent on β and ϵ, due to the restrictions in Inequation 3.3. They come with the time between exiting the last vehicle of trip i at index σ and boarding the next trip j at index σ': $\tau(d^j_{\sigma'}) - \tau(a^i_\sigma)$. Remember that for $\epsilon > 0$ this file also contains transfer arcs a, d for which the original timetable does not provide the mandatory minimum transfer time $\mathsf{walk}(a, d)$ or slightly exceeds the maximum waiting time β. To consider the benefit of a shifting, these originally not usable arcs are still included in the data. This way SERPENT is able to explore paths which can become viable in a well-chosen shifting.

6.2 Experimental Runtime

Figure 6.1 illustrates the computational performance, measured as the total Main-Step execution time, across the evaluated parameter space of ϵ and β. For Main-Step-Time total we took the mean out of three separate runs. The empirical data reveals a strict, monotonically increasing relationship between parameter relaxation and computational effort.

For any fixed maximum waiting time β, an increase in the relaxation parameter ϵ directly translates to a continuous growth in runtime. For instance, at $\beta = 15$, the computation time escalates from 13,45s at $\epsilon = 0$ to 19,79s at $\epsilon = 5$ (an increase of approximately 47.2%). This trend is inherently driven by the widening of the exploration threshold during the path-finding process, which forces SERPENT to evaluate a significantly larger sets of alternative path combinations.

Simultaneously, shifting to higher values of β lifts the baseline of the entire execution curve. At $\epsilon = 0$, changing β from 15 to 60 induces a 45.1% surge in computation time (from 13,45s to 19,51s). When both parameters are fully relaxed ($\epsilon = 5, \beta = 60$), the runtime peaks at 23,52s. This behaviour makes perfect sense, since allowing longer waiting times (β) and higher thresholds (ϵ) unlocks structurally distant OD pairs. Finding and validating these long-distance paths inherently expands the underlying search graph, explaining why SERPENT requires substantially more computational effort to terminate.

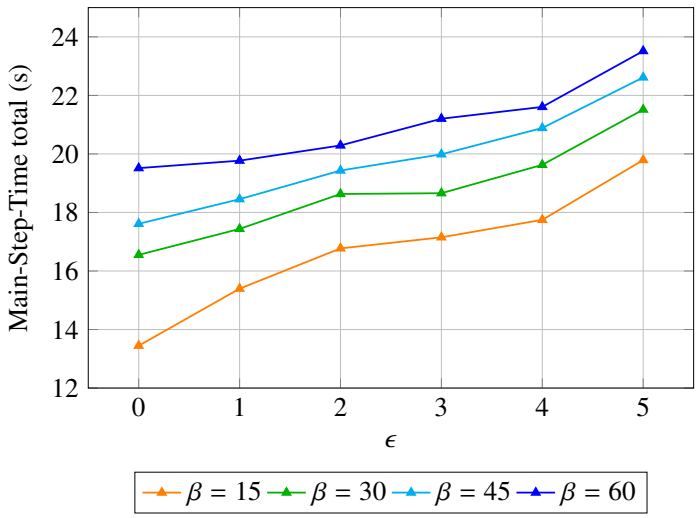


Figure 6.1: Total computation time (Main-Step-Time) in seconds for varying parameters of ϵ and β.

To accelerate the computation, a multithreaded version of SERPENT was implemented, which naively distributes the origin-zone and start-hour pairs (z, h) across the available kernels. Its computation time can be seen in figure 6.2. However, this approach does not account for workload balancing, meaning that variations in the complexity of explored path trees are ignored. Consequently, scaling performance encounters diminishing returns, and the execution time nearly stagnates beyond six threads.

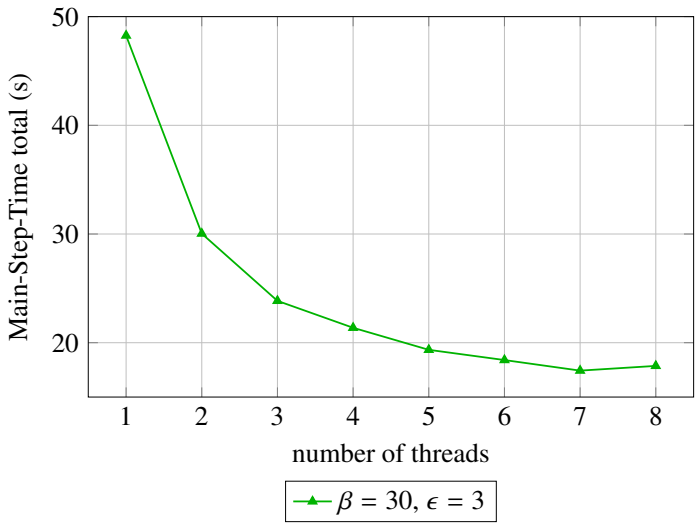


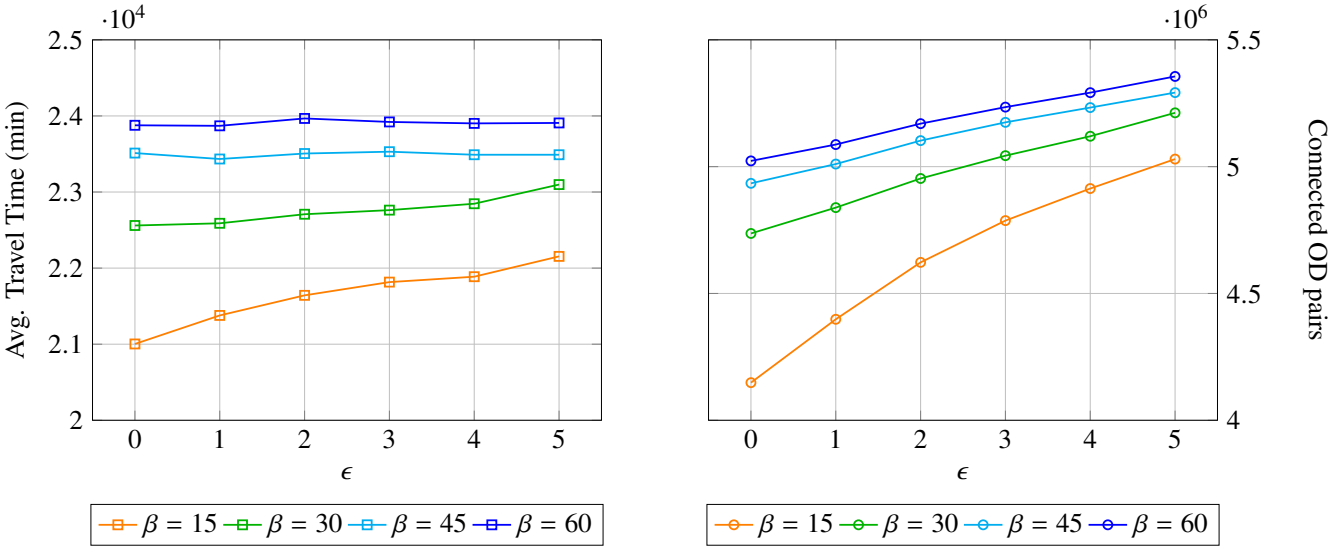
Figure 6.2: Total computation time (Main-Step-Time) in seconds for varying number of threads.

6.3 Average Travel Time

Figure 6.3 illustrates an interesting counter-intuitive phenomenon regarding the impact of relaxation parameters on both travel times and network connectivity.

As the maximum waiting time β increases, individual OD pairs that were already connected under stricter constraints do show an improvement in their respective travel times. However, the overall average travel time across the network paradoxically increases with larger values of β. This upward trend becomes even more pronounced as the relaxation parameter ϵ is further increased, particularly for lower values of β.

This behaviour can be attributed to a composition effect in the underlying dataset. While stricter constraints restrict the routing to a smaller subset of accessible paths, relaxing β and ϵ unlocks previously unreachable OD pairs. Critically, these newly discovered OD pairs frequently span larger physical distances across the network, as already mentioned in the previous Section 6.2, thereby inherently possessing significantly higher base travel times. Consequently, as the total number of discovered paths grows (as depicted in figure 6.3b), these newly introduced, long-distance paths disproportionately weigh down the average, ultimately shifting the overall mean travel time upward despite the localized routing optimizations.



(a) Average Travel Time

(b) Connected OD pairs

Figure 6.3: Comparison of average travel times and total number of connected OD pairs for different values of ϵ and β.

6.4 Comparison to other Algorithms

This Section contains important analysis of SERPENT. We break down how many paths are dropped due our speed-up techniques.

In Section 5.3 we already mentioned the limitations endured by limiting our transfers by a maximum waiting time β. Hence we designed SERPENT to focus on early arrivals at intermediate halts to speed up the path finding. We study the impact of our trade-off by comparing the path output of SERPENT to results of another one made within our project.

The algorithm we compare our results to, was designed to find not only the best, but the k best paths for capacity-demand calculations, hence we name it *K-Algorithm*. By setting $k = 1$, we extract one path with lowest travel time for each OD pair from a more complicated computation. But K-Algorithm also utilizes the same input files that form the basis of SERPENT, and is therefore limited to the same transfers induced by waiting time β and maximum shift ϵ. In our favour it does explore the whole search space $X(\beta)$, providing us with a set of optimal paths that we can compare our set of locally stepwise earliest paths to.

We designed an evaluation script comparing the path outputs for each combination of β and ϵ. It evaluates for each OD triple if the locally stepwise exploration can match the travel time of the K-Algorithm, or if it does not even find any path at all.

First, we give an overview of the general success rate of the stepwise earliest path exploration compared with the full solution space $X(0, \beta)$ and $X(\epsilon, 15)$. The evaluation results for the are presented in Figure 6.4 and Figure 6.5. Crucially, even under the most severely restricted

baseline configuration ($\epsilon = 0$ and $\beta = 15$), SERPENT successfully recovers identical travel times for 71.45% of all OD triples.

As the algorithmic constraints are relaxed, this success rate escalates drastically. For instance, maintaining $\epsilon = 0$ while increasing the threshold parameter β to 60 allows the match rate to reach 97.75%. A similar upward trajectory is visible when fixing $\beta = 15$ and gradually loosening ϵ from 0 to 5, which pushes the recovered OD triples to 93.62%.

The remaining fraction of unmatched OD triples represents highly rigid connections where identical travel times cannot be maintained under mild adjustments. Bridging these remaining gaps would require major structural transformations within the underlying timetable to prevent the necessity of intolerably long waiting times for passengers.

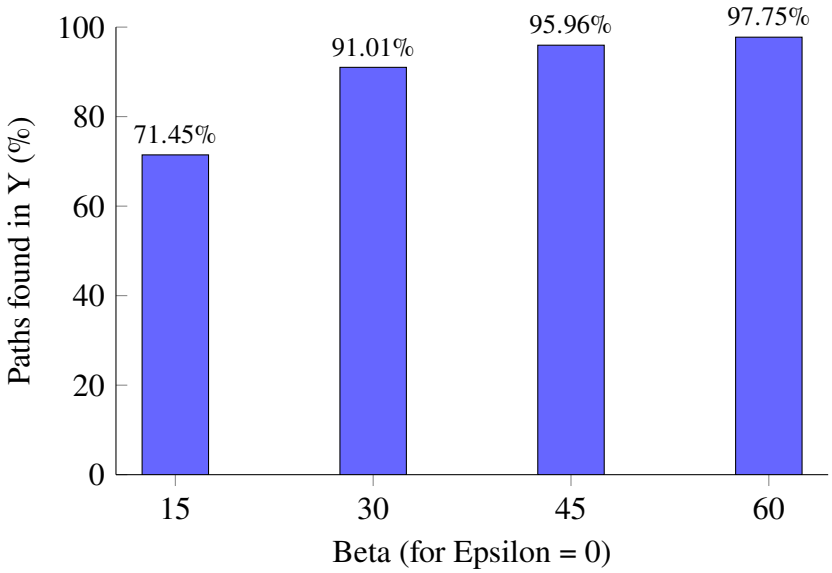


Figure 6.4: Percentage of paths from X that successfully got matched by Y for fixed $\epsilon = 0$ and increasing β.

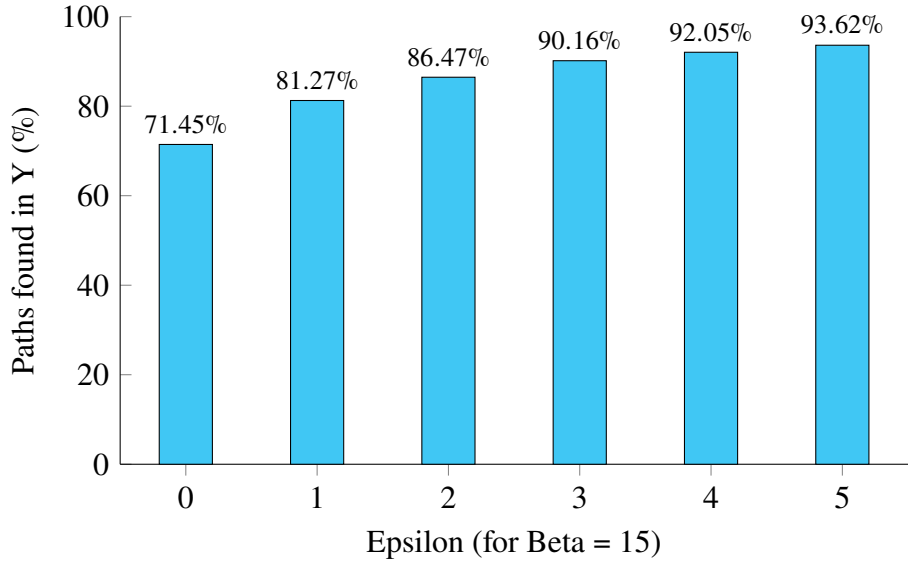


Figure 6.5: Percentage of paths from X that successfully got matched by Y for fixed $\beta = 15$ and increasing ϵ.

Figure 6.6 to 6.11 show evaluations for increasing β-values and a fixed ϵ. The reference data bar (called Ref) consists of all OD triples (z, z', h) for which a path was found in the most restrictive set of our database $X(\beta) \setminus Y(\beta)$ for $\beta = 15$ minutes. For these OD triples a path has been found by the K-Algorithm, but no path explored by our earliest locally stepwise pathfinding could match its travel time. Note that the y-axis are scaled differently, there are fewer initial differences (Ref) between the search spaces for a more relaxed shift limitation ϵ, as shown in the previous Figures 6.4 and 6.5. For each of these Ref-OD triples we check if our locally stepwise earliest method has improved accuracy for more relaxed waiting times β. The figures clearly show that for greater β, SERPENT misses substantially fewer paths.

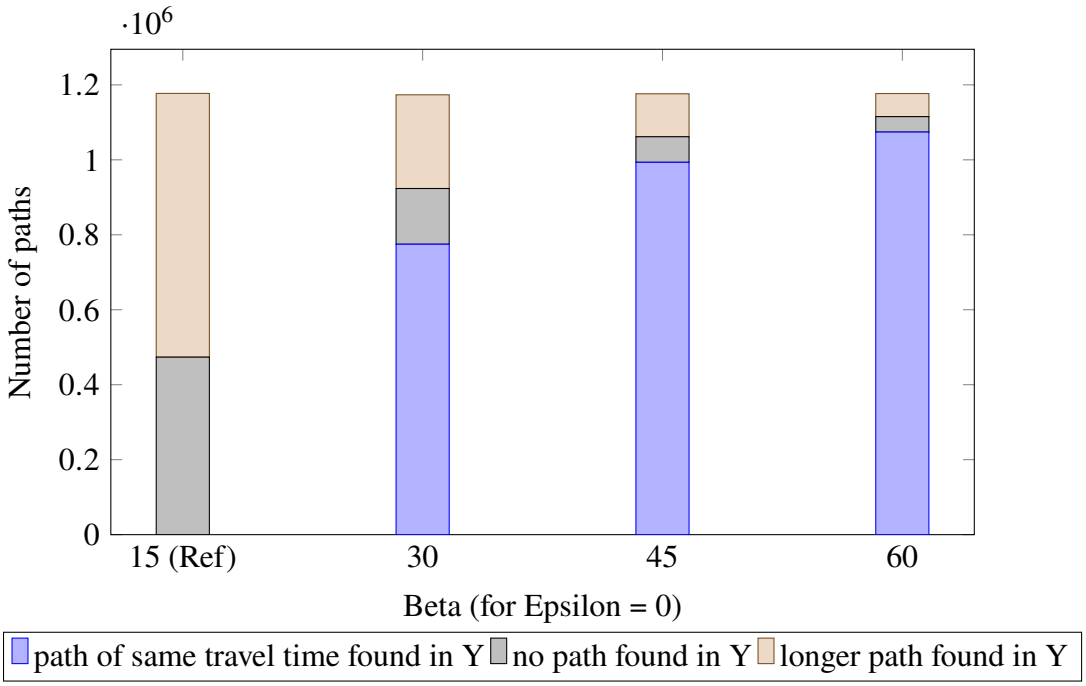


Figure 6.6: Statistic for increasing Beta-values and fixed Epsilon = 0

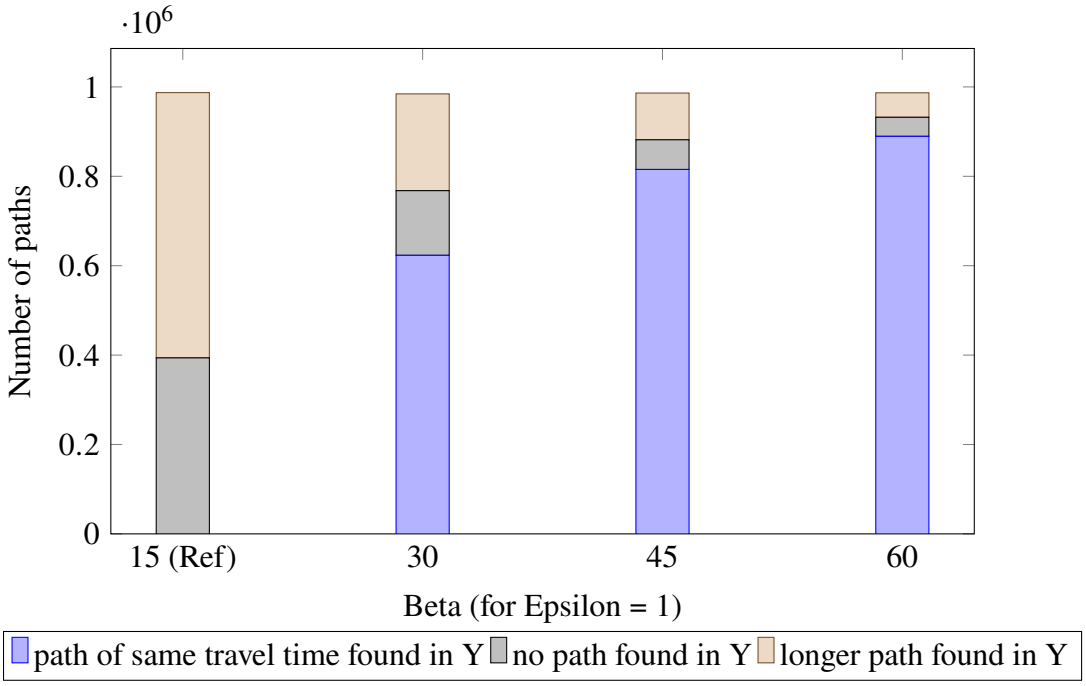


Figure 6.7: Statistic for increasing Beta-values and fixed Epsilon = 1

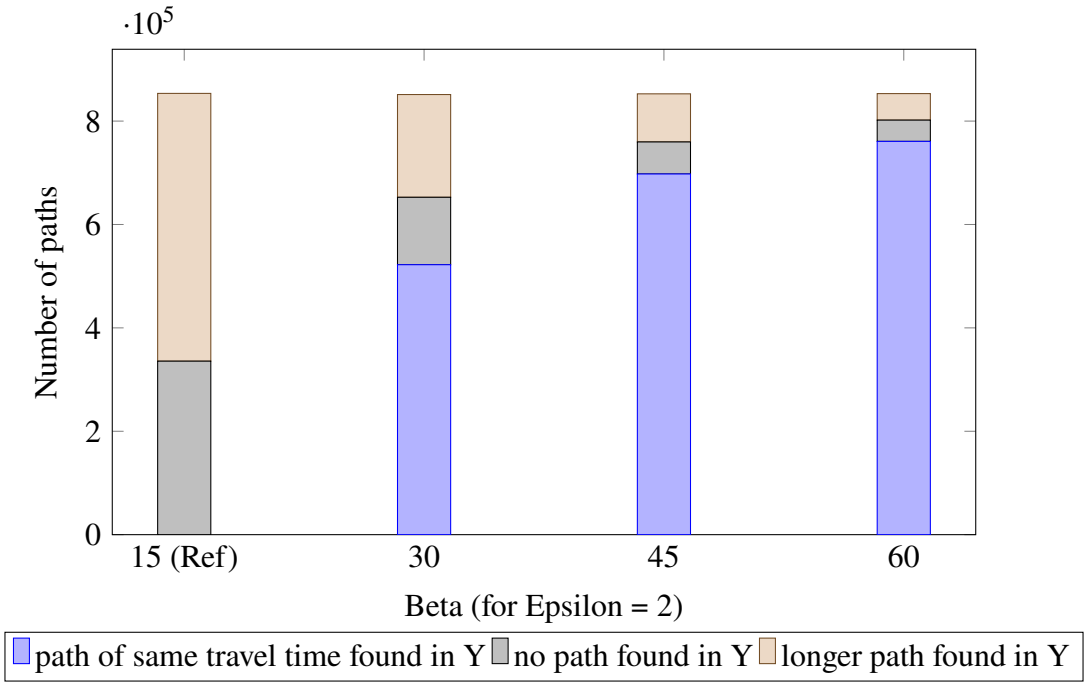


Figure 6.8: Statistic for increasing Beta-values and fixed Epsilon = 2

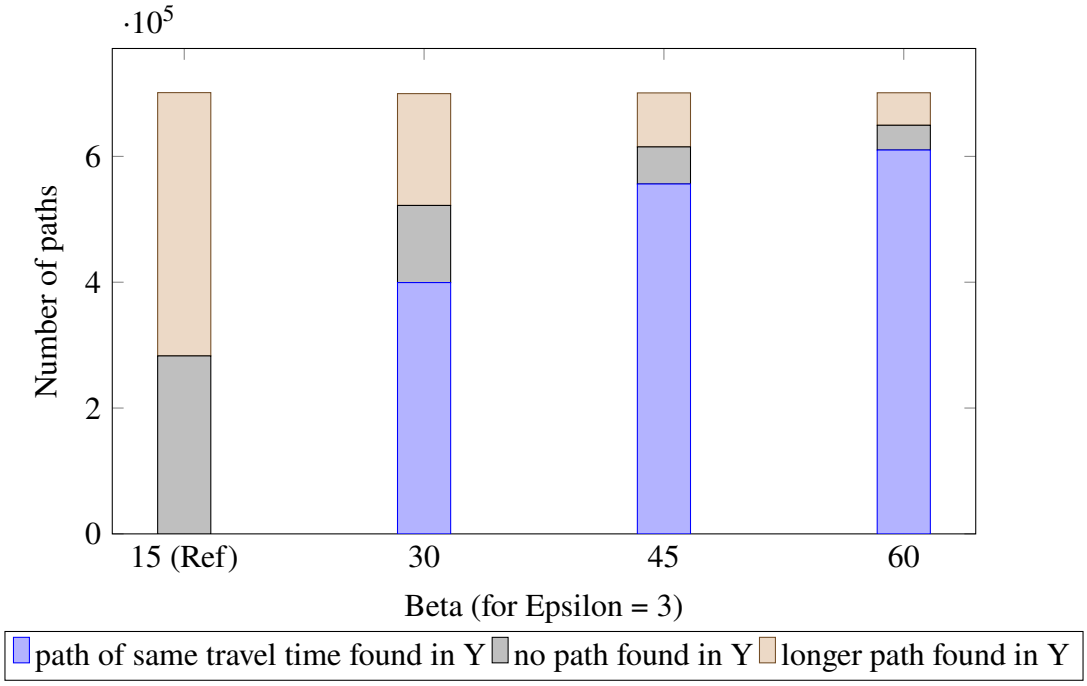


Figure 6.9: Statistic for increasing Beta-values and fixed Epsilon = 3

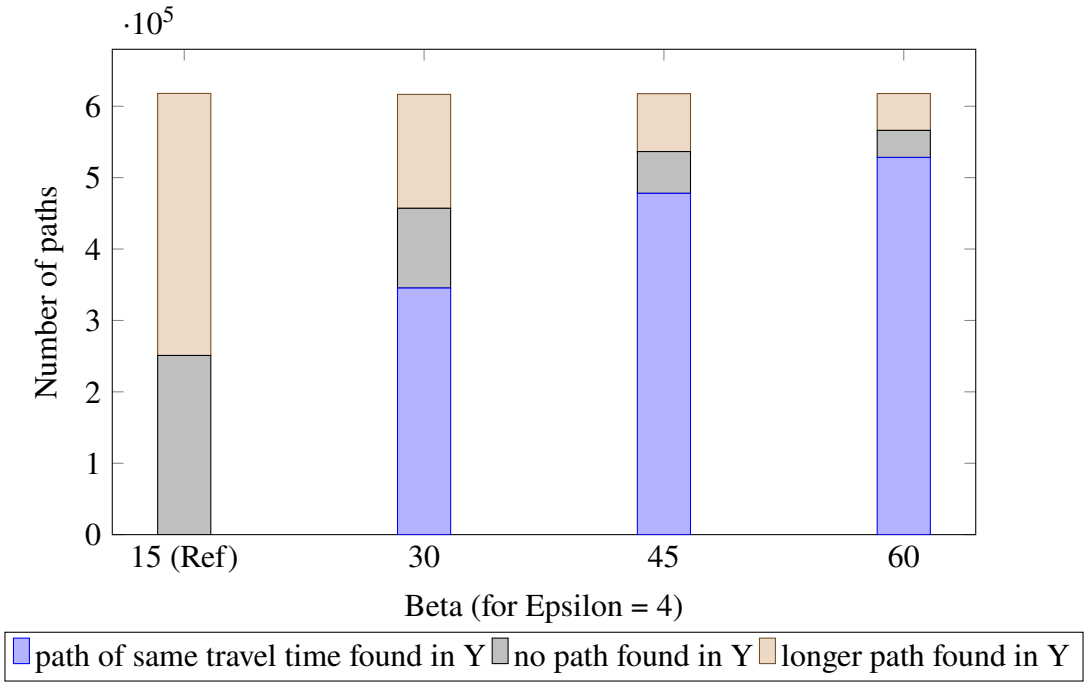


Figure 6.10: Statistic for increasing Beta-values and fixed Epsilon = 4

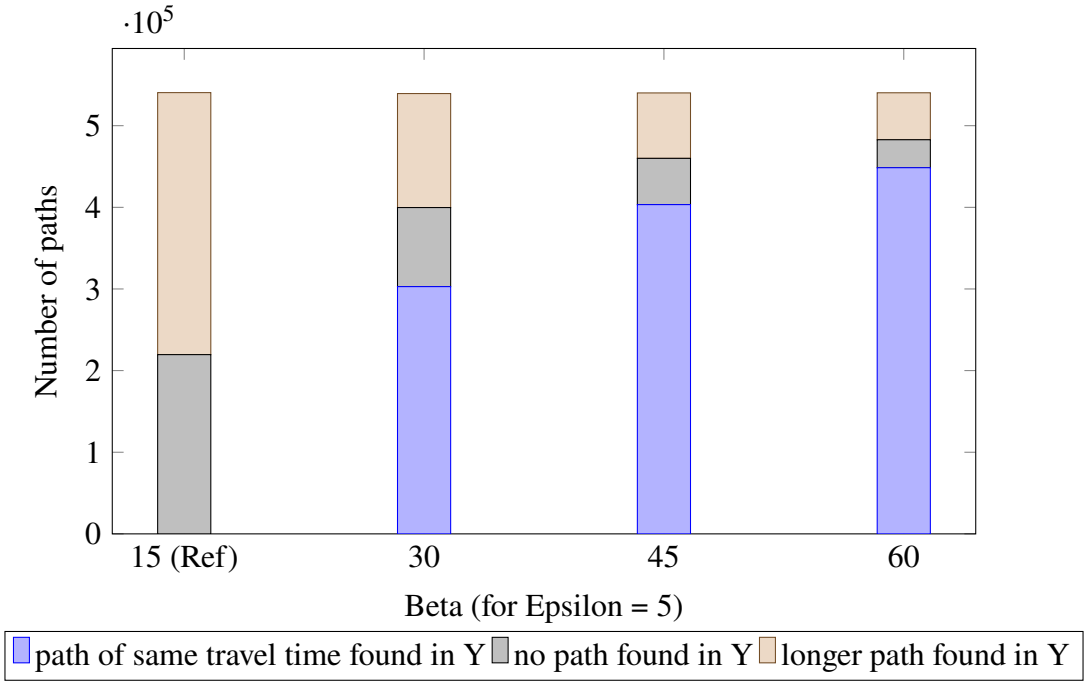


Figure 6.11: Statistic for increasing Beta-values and fixed Epsilon = 5

Similar results can be seen in Figures 6.12 to 6.15. In these evaluations β is fixed and our reference bar consists of the OD triples missed by SERPENT in the most restrictive setting $\epsilon = 0$. We can see that even a small possible time shift ϵ can have a great impact on the path finding. Remember that the paths found by SERPENT are not necessarily all valid simultaneously. In Section 3.3 we explain that our paths represent the maximum possible gain that a timeshift of ϵ can yield.

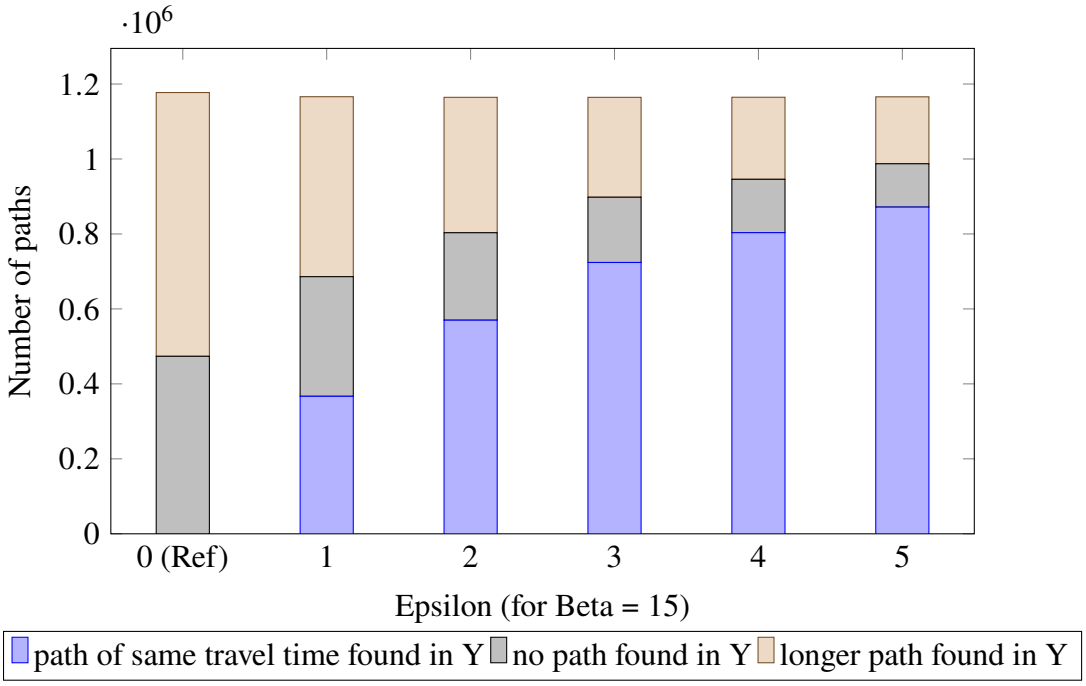


Figure 6.12: Statistic for increasing Beta-values and fixed Beta = 15

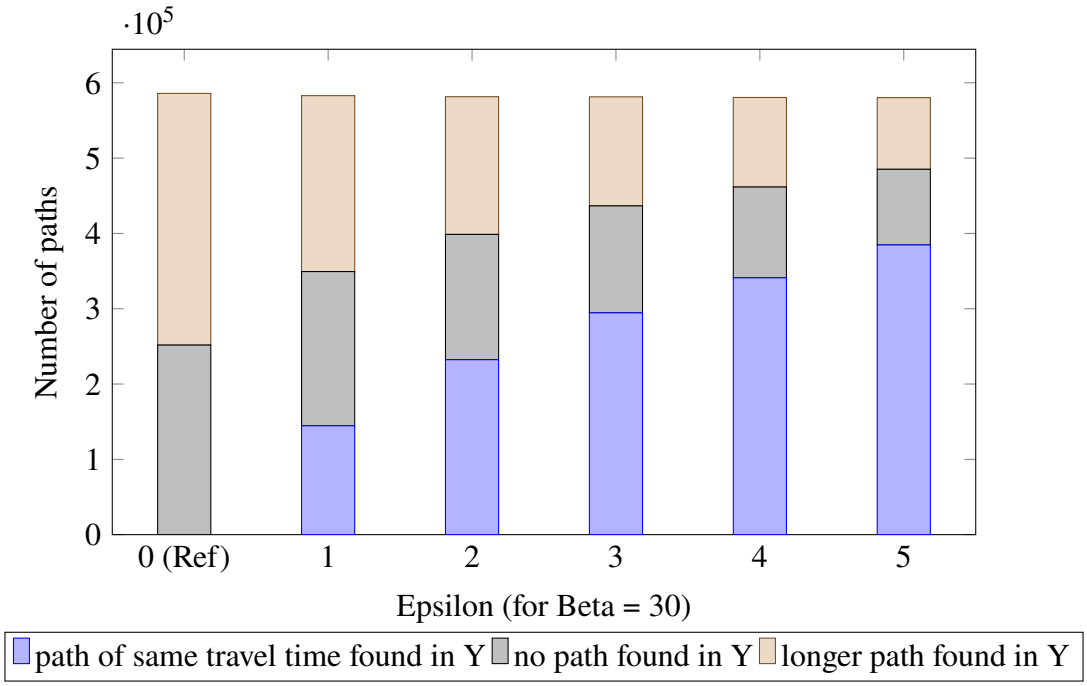


Figure 6.13: Statistic for increasing Beta-values and fixed Beta = 30

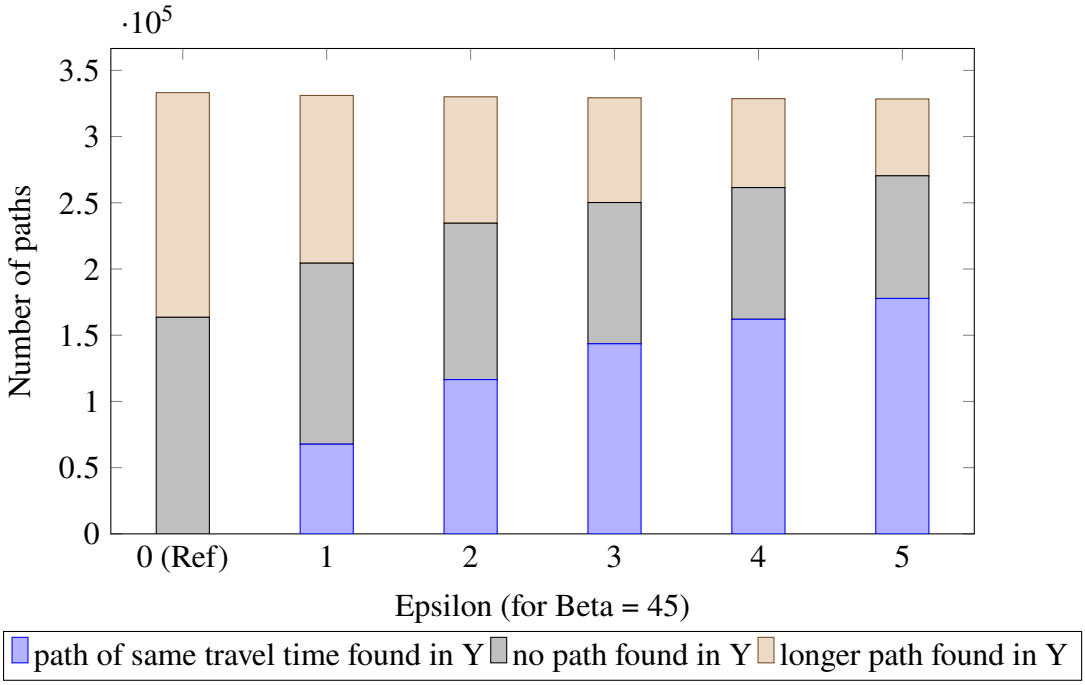


Figure 6.14: Statistic for increasing Beta-values and fixed Beta = 45

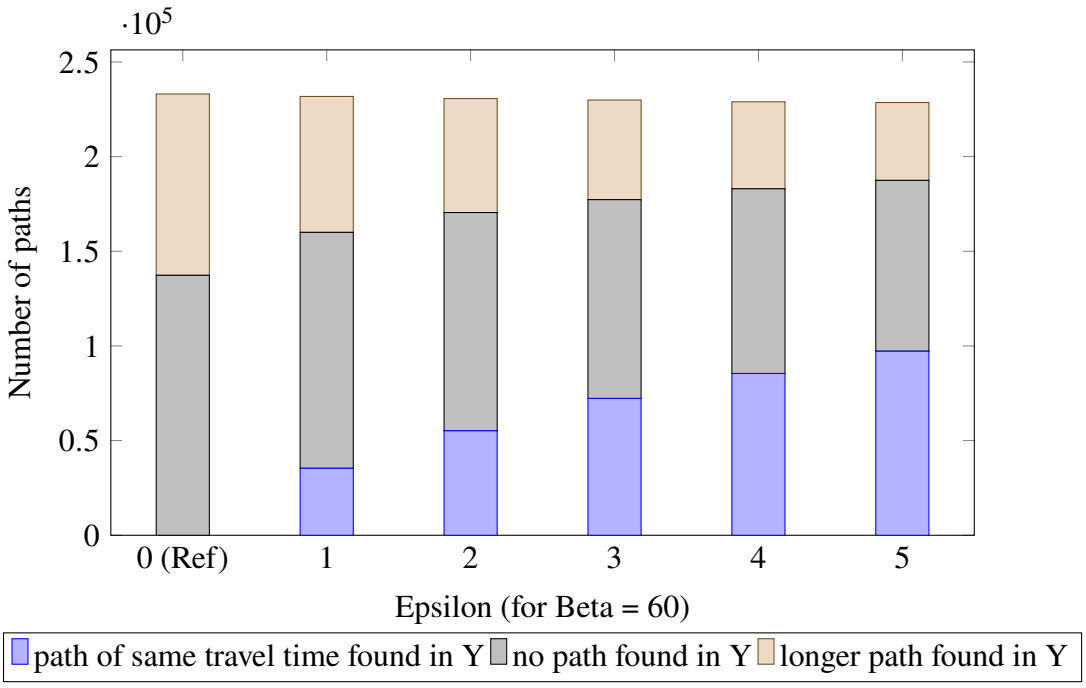


Figure 6.15: Statistic for increasing Beta-values and fixed Beta = 60

7 Conclusion and Outlook

We have established an algorithm SERPENT that successfully solves the Stepwise Earliest Arrival All-Pairs Shortest Path (SEAPSP) problem by systematically exploring locally stepwise earliest paths on a given timetable structure. As a primary result, the approach yields exactly one optimal path for each OD pair of zones (z, z'), which is guaranteed to be optimal among all locally stepwise earliest paths connecting z and z'.

While running SERPENT across a set of time periods $\mathcal{H}$ does not necessarily produce globally optimal graphs in terms of absolute travel time under all dynamic conditions, the generated data can be effectively utilized to indicate and diagnose structural weaknesses within the underlying public transit network and its associated timetable topology.

7.1 Summary of Contributions

The experimental and theoretical evaluations conducted in this thesis lead to several key insights:

1. **Algorithmic Efficiency:** The adoption of a RAPTOR-like exploration scheme tailored for each artificial start node d_z^{art} proves to be highly efficient and provides exceptional query performance for our specific real-world input datasets.

2. **Bi-Criteria Extension:** We demonstrated that upgrading the core framework in a manner similar to the range-RAPTOR (*rRAPTOR*) extension enables SERPENT to seamlessly handle bi-criteria travel times as an optimization objective, shifting the focus from a singular earliest arrival time to a broader Pareto-optimal perspective.

3. **Search Space Reduction via Pruning:** Restricting the edge or connection set by imposing a maximum tolerable waiting time β limits the explorable path set to a bounded subset $X(\beta)$. By furthermore inducing a station-based dominance pruning criterion, we are able to compress the search space even further, reducing it to a proper subset $Y(\beta) \subsetneq X(\beta)$ of the locally stepwise earliest explored paths.

7.2 Discussion of Path Quality and Structural Insights

Despite the aggressive pruning strategies employed to maintain computational feasibility, our empirical results indicate that the majority of optimal paths are still successfully preserved within the compressed search space $X^*(\beta)$.

Crucially, any optimal path P missed in the symmetric difference $X^*(\beta) \setminus Y^*(\beta)$ serving a given OD pair (z, z') exhibits a specific structural property: it can be functionally matched–in terms of identical total travel time–by a distinct compound path P. This alternative path is formed by the concatenation of two segments:

$$P' = P_Y \circ P_X,$$

where $P_Y \in Y(\beta)$ is a fully explored sub-path originating at d_z^{art} and terminating at an intermediate station s, and P_X represents the remaining trip segments of P from s to the artificial destination $a_{z'}^{\text{art}}$.

The necessity of transitioning from P_Y to P_X implies an intermediate waiting time τ^* at station s that strictly exceeds the threshold ($\tau^* > \beta$). Due to this prolonged transfer duration, such a connection is highly unlikely to be perceived as an attractive or viable option for actual passengers in a real-world scenario.

Consequently, the omission of path P from the final output does not represent a flaw in SERPENT. Instead, it serves as a valuable analytical diagnostic: by identifying the specific OD pairs where such paths are dropped, SERPENT directly indicates precisely which regions or connections within the public transit network are structurally deficient and should be prioritized for future infrastructural or scheduling improvements.

Bibliography

[1] Anil Bangia et al. “Modeling liquidity risk, with implications for traditional market risk management and mitigation”. In: *Journal of Banking & Finance* 26.2-3 (2002), pp. 317–333. DOI: 10.1016/S0378-4266(01)00225-8.

[2] Hannah Bast et al. “Route planning in transportation networks”. In: *Algorithm engineering: Selected results and surveys*. Springer, 2016, pp. 19–80.

[3] Richard Bellman. “On a routing problem”. In: *Quarterly of Applied Mathematics* 16.1 (1958), pp. 87–90.

[4] Richard Bellman and Robert Kalaba. “Dynamic programming and statistical communication theory”. In: *Proceedings of the National Academy of Sciences* 43.8 (1957), pp. 749–751.

[5] Thomas H Cormen et al. *Introduction to algorithms*. MIT press, 2022.

[6] Daniel Delling, Thomas Pajor, and Renato F Werneck. “Round-based public transit routing”. In: *Transportation Science* 49.3 (2015), pp. 591–604.

[7] Peter J Denning. “The locality principle”. In: *Communications of the ACM* 48.7 (2005), pp. 19–24.

[8] Julian Dibbelt et al. “Intriguingly simple and fast transit routing”. In: *International Symposium on Experimental Algorithms*. Springer. 2013, pp. 43–54.

[9] Edsger W. Dijkstra. “A note on two problems in connexion with graphs”. In: *Numerische Mathematik* 1.1 (1959), pp. 269–271. DOI: 10.1007/BF01386390.

[10] Ran Duan et al. “Breaking the sorting barrier for directed single-source shortest paths”. In: *Proceedings of the 57th Annual ACM Symposium on Theory of Computing*. 2025, pp. 36–44.

[11] Ricardo Euler, Niels Lindner, and Ralf Borndörfer. “Price optimal routing in public transportation”. In: *EURO Journal on Transportation and Logistics* 13 (2024), p. 100128.

[12] Lester R. Ford. *Network Flow Theory*. Tech. rep. P-923. RAND Corporation, 1956.

[13] Levin Hommel. “An Approach to Railway Connection Optimization”. MA thesis. FU Berlin, 2025.

[14] Donald B. Johnson. "Efficient algorithms for shortest paths in sparse networks". In: *Journal of the ACM (JACM)* 24.1 (1977), pp. 1–13. DOI: 10.1145/321992.321993.

[15] Rahul Kala. *On-Road Intelligent Vehicles: Motion Planning for Intelligent Transportation Systems*. Butterworth-Heinemann, 2016. ISBN: 9780128037300.

[16] Donald E Kirk. *Optimal control theory: an introduction*. Courier Corporation, 2004.

[17] Pedro Maristany de las Casas. "New Multiobjective Shortest Path Algorithms". PhD thesis. 2023.

[18] Ernesto Queiros Vieira Martins. "On a multicriteria shortest path problem". In: *European Journal of Operational Research* 16.2 (1984), pp. 236–245.

[19] John T. Moy. *OSPF: Anatomy of an Internet Routing Protocol*. Addison-Wesley Professional, 1998. ISBN: 9780201634723.

[20] Karl Nachtigall. "Time depending shortest-path problems with applications to railway networks". In: *European Journal of Operational Research* 83.1 (1995), pp. 154–166. DOI: 10.1016/0377-2217(95)00016-Q.

[21] Saul B. Needleman and Christian D. Wunsch. "A general method applicable to the search for similarities in the amino acid sequence of two proteins". In: *Journal of Molecular Biology* 48.3 (1970), pp. 443–453. DOI: 10.1016/0022-2836(70)90057-4.

[22] Luciana Operti, Günter Ratsch, and Alexander Zien. "Shortest path algorithms in computational biology: Analysis of metabolic and signaling pathways". In: *BMC Bioinformatics* 15.1 (2014), pp. 124–135.

[23] Yakov Rekhter and Tony Li. *A Border Gateway Protocol 4 (BGP-4)*. Tech. rep. RFC 1654. RFC Editor, 1994.

[24] Paolo Serafini. "Some considerations about computational complexity for multi objective combinatorial problems". In: *Recent Advances and Historical Development of Vector Optimization: Proceedings of an International Conference on Vector Optimization Held at the Technical University of Darmstadt, FRG, August 4–7, 1986*. Springer. 1987, pp. 222–232.

[25] Sascha Witt. "Trip-based public transit routing". In: *Algorithms–ESA 2015*. Springer, 2015, pp. 1025–1036. DOI: 10.1007/978-3-662-48350-3_85.

Declaration of Independent Work

I hereby declare to the Freie Universität Berlin that I have prepared the work submitted by me independently, without the use of any aids other than those explicitly permitted in advance and indicated by me. Passages taken verbatim or in essence from other works have been marked as such, indicating the source. The present work has not been submitted for any other examinations. I am aware,

- that I will not have passed this examination if I fail to meet the submission deadline for my written work known to me,
- that in the case of attempted deception I will not have passed this examination,
- that in the case of a serious attempt at deception, I may have definitively failed the overall examination and may no longer continue my studies in this degree program, and
- that if I have used AI-based tools to produce this work, I bear responsibility for any erroneous or biased content generated by AI, incorrect references, violations of data protection and copyright law, or plagiarism.

Berlin, July 6, 2026

Milan Möllenberg (4360289)